MDPI

Article

Enhancing Consent Policy Privacy in Blockchain-Based Healthcare Data Sharing

Kun Li [1,*], Ankur Lohachab [1], Michel Dumontier [1] and Visara Urovi [1]

[1] Department of Advanced Computing Sciences, Maastricht University, 6200 MD Maastricht, The Netherlands; ankur.lohachab@maastrichtuniversity.nl (A.L.); michel.dumontier@maastrichtuniversity.nl (M.D.); v.urovi@maastrichtuniversity.nl (V.U.)

* Correspondence: k.li@maastrichtuniversity.nl

Abstract

In blockchain-based healthcare data sharing, consent policies are expected to be enforced verifiably and auditably, but public-chain transparency may expose sensitive policy information through on-chain representations, verification interfaces, and execution artifacts. Existing privacy-preserving solutions mainly protect data contents, identities, or transactions, not the governing consent policies. We propose the Trusted Proof Service-assisted Zero-Knowledge Policy Interpreter (TPS-ZKPI), which lets an on-chain verifier check authorization decisions while limiting leakage about policy content, identity, and structure. TPS-ZKPI combines fixed-length policy encoding, a reusable zero-knowledge policy interpreter, commitment-based policy binding, and public-chain transcript normalization; the proof binds the program length and the authorization context, and profile-backed attributes are derived in-circuit from an issuer-signed credential. We evaluate TPS-ZKPI on Polygon Amoy against metadata-only public-chain adversaries with supervised and unsupervised inference attacks. Under a multi-seed leakage criterion, no attack recovers hidden policy information reproducibly above chance, except two tasks whose separability is carried entirely by the randomized gas-padding draw; a paired ablation attributes the gas-channel protection to that padding, at an overhead of about 46% on access and 38% on commit gas.

Keywords: blockchain; consent policy privacy; zero-knowledge proofs; Data Use Ontology (DUO); healthcare data sharing; metadata leakage

1. Introduction

Blockchain technology has emerged as a promising foundation for decentralized and auditable healthcare data sharing [1–3]. Its transparency and immutability enable verifiable consent enforcement and traceable authorization decisions, supporting cross-institutional accountability. At the same time, consent enforcement on public blockchains may expose policy-dependent contextual information, even when the underlying health data remain encrypted [4,5].

In healthcare data sharing, a *consent policy* encodes the conditions under which a data subject authorizes others to access or use their health data, such as who may access the data, for what purpose, and under what constraints. Consent policies in healthcare are often privacy-sensitive [6]. For example, a policy based on biomedical data-use terms may reveal whether a dataset is restricted to disease-specific research, whether ethics approval is required, or whether access is limited to particular institutions or projects. If such policies are directly represented in smart contracts, embedded in verification logic, or reflected

Received:
Accepted:
Published:

Copyright: © 2026 by the authors. Submitted to *Blockchains* for possible open access publication under the terms and conditions of the Creative Commons Attribution (CC BY) license.

in public execution traces, unauthorized observers may infer information about the protected resource, the data subject's constraints, or the requester's authorization context.

Existing privacy-preserving blockchain solutions mainly protect data contents, user identities, or transaction details [7–9]. Encrypted data-sharing mechanisms such as attribute-based encryption and proxy re-encryption protect access to data [10,11], while zero-knowledge transaction systems hide asset ownership, values, or participant relationships [12]. None of these mechanisms protects the confidentiality of the policies that govern access decisions. Even mechanisms designed to hide policy content, such as hidden-policy attribute-based encryption [13], do not control the policy-dependent artifacts that a public chain exposes while enforcing the policy. In blockchain-based consent management, policy leakage may arise not only from explicit policy disclosure, but also from policy-dependent artifacts such as calldata size, public-input layout, emitted events, verifier interfaces, and gas usage. Therefore, policy-private consent enforcement requires more than hiding data or proving correctness in zero knowledge; it also requires controlling what the public-chain transcript reveals about the hidden policy. This paper studies this problem for blockchain-based healthcare data sharing under public-chain observability. The central challenge is to enable an on-chain verifier to check that a requester satisfies a provider-defined consent policy while ensuring that metadata-only public-chain observers learn minimal information about the governing policy and the requester's authorization evidence from the resulting verification transcript. We focus on Boolean-based consent policies, including policies derived from biomedical data-use conditions such as terms from the Data Use Ontology (DUO), the GA4GH standard vocabulary of consent-based data-use conditions [6], and disease-scope constraints [14].

We address this problem with the Trusted Proof Service-assisted Zero-Knowledge Policy Interpreter (TPS-ZKPI). In TPS-ZKPI, the data provider (who can be seen as the data owner) commits to a hidden consent policy on-chain and provisions the corresponding policy-bound proving material to a Trusted Proof Service (TPS). The requester submits policy-relevant authorization evidence, such as attributes, credentials, and access context, to the TPS. The TPS constructs the private witness and generates a zero-knowledge proof showing that the requester satisfies the committed policy. The proof is then verified on-chain against the provider's policy commitment. This yields an auditable authorization outcome without exposing the policy logic to the public-chain transcript.

TPS-ZKPI combines the following design elements. First, it uses TPS-assisted proof generation to keep confidentiality-sensitive policy material and requester evidence outside the public-chain transcript. Second, it compiles heterogeneous Boolean-based consent policies into fixed-length Reverse Polish Notation (RPN) programs. Third, building on the standard commit-and-prove paradigm, it evaluates these programs through a reusable zero-knowledge policy interpreter that avoids policy-specific verifier contracts and supports policy evolution without changing the public verification interface. Fourth, it applies observability-aware transcript normalization to reduce policy-dependent variation in calldata, logs, public parameters, and gas-related behavior.

We evaluate TPS-ZKPI using attack-driven empirical analysis under a metadata-only threat model. The evaluation operationalizes policy privacy through three objectives: Policy Content Hiding (PCH), Policy Indistinguishability (PI), and Structure and Parameter Privacy (SPP). Supervised and unsupervised attacks test whether observable public-chain artifacts allow adversaries to recover hidden policy content, distinguish otherwise-equivalent policies, or infer structural and parameter properties. The results show that, after transcript normalization, no modeled attack recovers hidden policy information reproducibly above chance under a multi-seed leakage criterion. A paired ablation serves as a positive control: when gas padding is removed, the same attackers recover policy-

dependent signal, and that signal collapses under the full mechanism, isolating randomized gas padding as the defense responsible for closing the gas channel.

This paper makes the following contributions:

- We propose TPS-ZKPI, a TPS-assisted zero-knowledge approach for publicly verifiable and auditable consent enforcement, in which on-chain proof verification and auditability hold without trusting the TPS, while policy confidentiality and faithful witness construction are conditional on an explicit, delimited trust assumption—keeping policy-bound proving information and requester authorization evidence off-chain.
- We develop a reusable zero-knowledge policy interpreter for fixed-length Boolean RPN policy programs over a bounded token alphabet, combining policy commitment binding with a policy-independent on-chain verification interface that serves all policies through one circuit and supports policy evolution without changing the on-chain verifier.
- We provide an attack-driven empirical evaluation of residual policy-dependent leakage from normalized public-chain metadata on the Polygon Amoy testnet[1], using supervised classification and unsupervised clustering attacks under a multi-seed permutation-test criterion. A paired gas-padding ablation then isolates randomized gas padding as the defense responsible for closing the gas channel and quantifies its gas-cost overhead.

The remainder of this paper is organized as follows. Section 2 reviews the technical foundations: cryptographic access control in blockchain healthcare, zero-knowledge proofs, and the Data Use Ontology. Section 3 discusses related work and positions TPS-ZKPI against existing blockchain consent-management, privacy-preserving blockchain, hidden-policy, private policy-evaluation, reusable proving, and metadata-leakage approaches. Section 4 defines the system and trust model, the public-chain adversary, and the privacy objectives. Section 5 presents the TPS-ZKPI design. Section 6 reports the empirical evaluation. Section 7 discusses limitations and future work, and Section 8 concludes the paper.

2. Background

This section reviews the technical foundations relevant to policy-private consent enforcement on blockchains. We first introduce the cryptographic access-control mechanisms commonly used in blockchain-based healthcare data sharing. We then review zero-knowledge proofs and the Data Use Ontology (DUO) [15], which together provide the basis for the policy representation and verification mechanisms used in our approach.

2.1. Cryptographic Access Control in Blockchain Healthcare

Fine-grained consent management in blockchain-based healthcare data sharing is commonly built on cryptographic access-control primitives, chiefly Ciphertext-Policy Attribute-Based Encryption (CP-ABE) and Proxy Re-Encryption (PRE) [11,16,17]. In CP-ABE, the encrypting party embeds an attribute-based access policy into the ciphertext, so that only requesters whose credentials satisfy the policy can decrypt the protected data [10]. PRE lets a semi-trusted intermediary transform a ciphertext encrypted for one party into one decryptable by a designated requester, delegating access without exposing the plaintext [17]. Both primitives remain the building blocks of current blockchain healthcare designs: recent schemes combine PRE with on-chain records to delegate access to medical records and IoMT data [18,19], and CP-ABE variants with hidden policies and attributes, distributed key generation, or multi-authority hierarchies address the exposure

[1] https://amoy.polygonscan.com

of the access structure itself [20–22]. We cite the original constructions because the later systems inherit their trust and disclosure properties, which is what matters for this paper: these primitives establish who may obtain the data, and how they relate to *policy* confidentiality on a public chain is discussed in Section 3.

2.2. Zero-Knowledge Proofs

Zero-knowledge proofs (ZKPs) are cryptographic protocols that allow a prover to convince a verifier that a statement is true without revealing any information beyond the validity of the statement [23]. Modern ZKP systems support verifiable computation over private inputs, so that complex predicates can be proven without disclosing the underlying data. In blockchain settings, ZKPs are widely used to reconcile public verifiability with privacy: privacy-focused systems such as Zerocash and Tornado Cash apply them to conceal transaction details or provide transaction anonymity [9,24]. These properties make ZKPs a natural building block for proving that a requester satisfies a hidden consent policy, which is the direction our approach develops.

2.3. The Data Use Ontology

The Data Use Ontology (DUO) provides a standardized vocabulary for expressing biomedical data-sharing consent conditions, and its terms are widely adopted in genomics and biobanking to enable semantic interoperability [6,15]. A DUO-based policy combines permissions and modifiers—for example, general research use (GR), disease-specific research (DS), or a requirement that ethics-committee approval be obtained (IRB)—into the condition under which access is granted.

DUO Term Categories

Although DUO offers a rich set of permissions and modifiers, its terms are heterogeneous in how they must be evaluated for compliance. To structure their handling in a zero-knowledge setting, we group DUO terms into four categories:

- **Boolean-only:** simple yes/no conditions that can be represented directly without external validation, e.g. “no restriction” (NRES), general research use (GR), or “health/medical/biomedical research” (HMB).
- **Ontology-linked:** terms whose evaluation involves consistency checks against external vocabularies or classifications, e.g. disease-specific research (DS), population origins or ancestry research (POA), or clinical care use (CC).
- **Obligation-required:** terms that impose procedural or temporal obligations, e.g. “publication required” (PUB), “publication moratorium” (MOR), “time limit on use” (TS), or “return to database” (RTN).
- **Credential-required:** terms whose evaluation relies on attested proof of requester status or approvals, e.g. “collaboration required” (COL), “ethics approval required” (IRB), “project-specific restriction” (PS), or “institution-specific restriction” (IS).

Table 1 summarizes representative DUO terms under this classification. Some terms reduce to simple Boolean flags, while others rely on ontology references, external obligations, or credential attestations. Later sections describe how these heterogeneous requirements are harmonized into a unified privacy-preserving verification process.

3. Related Work

Blockchain-based consent management for healthcare data sharing has been studied extensively, with smart contracts used to record, enforce, and audit consent across institutions [1–3]. This line of work focuses on making consent decisions verifiable and auditable, and the privacy mechanisms surveyed in this space mainly protect health data contents

Table 1. Representative DUO terms grouped by evaluation category.

Category	Example Terms	Description
Boolean-only	NRES (DUO:0000004), GR (DUO:0000042), HMB (DUO:0000006)	Direct yes/no conditions, no external validation required.
Ontology-linked	DS (DUO:0000007), POA (DUO:0000011), CC (DUO:0000043)	Evaluated against external ontologies such as ICD-10 or MONDO.
Obligation-required	PUB (DUO:0000019), MOR (DUO:0000024), TS (DUO:0000025), RTN (DUO:0000029)	Impose obligations such as publication duties, time limits, or data return.
Credential-required	COL (DUO:0000020), IRB (DUO:0000021), PS (DUO:0000027), IS (DUO:0000028)	Depend on credentials, approvals, or institutional/project-specific restrictions.

and user identities [5]. The confidentiality of the governing consent policy—and what public-chain enforcement artifacts reveal about it—is typically out of scope.

Beyond the consent-management setting, existing work addresses important fragments of policy-private consent enforcement, but not their combination under public-chain observability. Encrypted data-sharing mechanisms, such as Ciphertext-Policy Attribute-Based Encryption (CP-ABE) and proxy re-encryption (PRE) [16,17], control access to encrypted data. However, standard CP-ABE attaches the access policy to the ciphertext in the clear, exposing the policy itself, while PRE-based delegation may reveal authorization patterns that data encryption is not designed to hide. Transaction-privacy systems, including Zerocash-style protocols [24] and Ethereum-oriented systems such as ZETH and AZTEC [7,8], provide public verification over private transaction witnesses, but they mainly target asset ownership, values, or participant relationships rather than compliance with a provider-defined consent policy.

Hidden-policy access-control work reduces direct policy exposure by using encrypted policies, hidden-policy ABE variants, off-chain matching, or policy-holder components [13,20–22]. Functional encryption gives a more general cryptographic abstraction for revealing function outputs while hiding inputs [25], but it does not by itself yield publicly verifiable authorization evidence. Private-state blockchain systems, including Secret Network and Oasis/Ekiden, protect smart-contract state or execution through encrypted state and trusted execution environments [26,27]. These lines of work address adjacent forms of policy or state hiding, but TPS-ZKPI targets a more specific setting: hiding consent policies on a public chain while retaining public verifiability of each authorization decision. Even where policies are hidden, existing designs may still leak policy-dependent artifacts through ciphertext structures, policy-update transactions, authorization workflows, or repeated interactions. Moreover, many target permissioned or off-chain settings, where enforcement artifacts are not globally visible as they are on public blockchains.

Self-sovereign-identity (SSI) architectures move authorization to verifiable claims held by the requester and shift the question to whom the holder may trust with those claims. Buccafurri et al. [28] combine smart contracts with ciphertext-policy attribute-based proxy re-encryption so that a holder's claims can be read only by verifiers that satisfy a holder-defined policy, with the contract keeping verifiers accountable for the access they obtain; zero-knowledge presentations of credentials likewise let a patient authenticate anonymously against public or private chains [29], and surveys of zero-knowledge

proofs for blockchain privacy catalogue this design space [12,30]. TPS-ZKPI sits on the other side of the same authorization relation. In SSI designs the protected object is the requester's claim set and the policy that governs it is the holder's, whereas here the protected object is the provider's consent policy: the requester's claims are consumed as private witnesses of a proof whose statement is fixed and whose verifier is a public contract, so verifier authorization is not a design question (any observer may verify) and accountability is carried by the recorded, context-bound outcome rather than by a contract that meters the release of claims. The trust the two settings place off-chain also differs: an SSI verifier that decrypts a claim learns it, whereas the TPS learns both policy and claims, which is why Section 4 treats the TPS as an explicit trust boundary and Table 3 states what survives its compromise. Combining the two—holder-controlled release of the claims that TPS-ZKPI consumes as witnesses—is a natural extension and would remove the TPS's view of the requester evidence, as discussed in Section 7.

Privacy-preserving access-control and policy-evaluation mechanisms have been studied in credential-based and XACML/SAML settings [31], as well as in multi-party policy evaluation using homomorphic encryption and secure function evaluation [32]. These approaches support confidential authorization computation, but they are not primarily designed for settings where policy-evaluation artifacts—calldata, public inputs, emitted events, timing, and gas usage—are permanently and globally visible on a public blockchain. In addition, these approaches do not directly provide compact evidence that a smart contract can verify against a committed, provider-defined policy instance.

Reusable proving systems, including universal-setup SNARKs, composable proof systems, and zkVMs [33–35], improve maintainability by reducing the need for policy-specific circuits or verifier contracts. However, when deployed for public-chain consent enforcement, reusable proving alone does not guarantee policy privacy: policy-dependent program representations, public inputs, calldata layouts, logs, or gas profiles may still remain observable. This motivates TPS-ZKPI's use of a domain-specific reusable policy interpreter, which preserves proof reuse while exposing only a policy-independent public verification interface.

Public-chain metadata-leakage studies show that cryptographic privacy guarantees may be weakened by observable transaction-level artifacts such as timing and volume patterns [4]. Techniques such as batching, decoys, and traffic shaping can reduce some observable signals [36], but they are usually designed for payment privacy, anonymity sets, or transaction-linkability settings. They do not directly provide hidden-policy compliance verification or auditable consent enforcement. In contrast, TPS-ZKPI treats metadata leakage as part of the policy-privacy objective rather than as a separate transaction-level concern.

Existing approaches address verifiable consent recording, data confidentiality, transaction privacy, policy hiding, confidential policy evaluation, proof reuse, or metadata mitigation in isolation, whereas policy-private consent enforcement under public-chain observability requires the combination of policy confidentiality, metadata-leakage control, verifiable auditability, and a stable verification interface for evolving Boolean-based consent policies. Table 2 summarizes this positioning.

4. Problem and Threat Model

4.1. System and Trust Model

We consider a blockchain-based healthcare data-sharing system in which two types of users interact to share data: 1. data providers, who provide protected resources and govern access to them through consent policies, and 2. data requesters, who seek access to these resources subject to those policies. TPS-ZKPI mediates this interaction through three

Table 2. Positioning of representative approaches against policy-private consent enforcement under public-chain observability (Yes / Partial / No; assumptions in parentheses are part of the claim).

Approach	Policy confidentiality	Metadata-leakage mitigation	Verifiable auditability	Main limitation
Blockchain consent management [1,2]	No	No	Yes	Records and enforces consent verifiably on chain, but policy confidentiality and policy-dependent leakage are typically out of scope.
Data / transaction privacy [7,8,16,17,24]	No	Partial	Partial	Protects encrypted data access or transaction semantics, but not the governing consent policy or policy-dependent authorization workflow.
Hidden-policy access control [13,22]	Partial	Partial	Partial	Reduces direct policy exposure, but may leak through workflows, updates, or policy-holder interactions.
MPC / HE policy evaluation [31,32]	Yes	No	Partial	Supports confidential computation, but usually lacks public-chain-verifiable authorization evidence.
Universal SNARKs / zkVMs [33–35]	No	Partial	Yes	Supports proof reuse and public verification, but does not by itself hide policy-dependent public artifacts.
Metadata-leakage defenses [36]	No	Yes	No	Models or mitigates observable signals, but does not provide hidden-policy consent enforcement.
TPS-ZKPI	Yes (under TPS trust)	Yes (modeled channels)	Yes	Provides policy-private, metadata-aware, verifiable consent enforcement under the stated TPS trust assumption.

components: a TPS operating off-chain, an on-chain access-control manager (ACM), and a data access layer. The provider defines a consent policy P for a protected resource on behalf of the data subject whose consent it encodes, and registers an on-chain commitment h to the encoded policy. The requester seeks access by submitting policy-relevant authorization evidence, such as attributes, credentials, and access context. The TPS constructs the private witness and generates a zero-knowledge proof of policy compliance. The ACM verifies the proof on-chain and records the authorization outcome, while the data access layer grants access according to the recorded decision.

TPS-ZKPI relies on the following assumptions. First, policies are provider-controlled: the provider is responsible for defining the applicable consent policy and committing the corresponding policy instance on-chain. Second, the TPS is trusted to preserve the confidentiality of policy-bound proving material and requester-submitted authorization evidence, and to follow the prescribed proof-generation workflow. Third, the commitment scheme is assumed to be hiding and binding, and the zero-knowledge proof system is assumed to satisfy completeness, soundness, and zero knowledge with respect to the implemented policy-compliance relation. Credential issuance, revocation, and real-world identity verification are treated as external functions.

Conceptually, the TPS is an independent proof service, distinct from the provider; this separation is what makes relocating trust meaningful, since it prevents the provider from being the single party that holds both the policy and the requester evidence. Possible instantiations of such an independent service include a third-party proof service, a TEE-backed proof service that makes the trust assumption hardware-attested, or MPC/collaborative SNARK proof generation in which no single party sees both policy material and requester evidence.

For implementation and evaluation simplicity, the current evaluation co-locates the TPS with the provider (a provider-operated TPS). This co-location concentrates trust at the provider side: a single party now holds both the policy and the requester evidence, so a compromised provider-operated TPS could substitute inputs whose origin is not in-

dependently authenticated or withhold proof generation from legitimate requesters. Two guarantees must be distinguished here. Authorization correctness is primarily enforced by the commitment and on-chain proof verification: a proof that does not satisfy the committed policy relation is rejected, largely independently of how the TPS is instantiated. Policy confidentiality, in contrast, is not cryptographically enforced against the TPS itself; it rests entirely on the trust assumption stated above. Our evaluation therefore assumes a TPS that follows the prescribed workflow and isolates the remaining question: what a public-chain observer can infer from the resulting public artifacts. Weakening the TPS trust assumption itself is orthogonal and is the role of the reduced-trust instantiations above. Table 3 summarizes, for each party or failure mode, which of the four guarantees—policy confidentiality, authorization correctness, liveness, and accountability—is enforced cryptographically, which holds only under the TPS trust assumption, and which the prototype leaves to external functions or does not provide.

Table 3. What each guarantee of TPS-ZKPI rests on, per threat. C: cryptographic (commitment binding, proof soundness and zero knowledge, credential signature, on-chain checks); T: trust in the TPS following the prescribed workflow; E: external or lifecycle function; –: not provided by the prototype.

Threat	Policy confidentiality	
Passive public-chain observer	C (proof, commitments) + empirical (gas padding)	
Requester (untrusted)	C: sees only the public view	C: cannot forg
Honest-but-curious TPS	T: TPS holds plaintext policy and evidence	
Compromised or colluding TPS	–: policy and evidence exposed	C for committed
Provider-operated TPS (this evaluation)	T, concentrated at the provider	
Forged or unauthorized issuer	n/a	C against forgery under the

Attribute-origin guarantees depend on the evidence type. For profile-backed DUO terms, the prototype verifies an EdDSA-Poseidon credential under a provider-selected, policy-bound issuer-key commitment. The credential binds the policy commitment, requester pseudonym, protected resource, claims, validity interval, and credential identifier; the corresponding policy attributes are derived inside the circuit. Thus, subject to signature unforgeability and Poseidon collision resistance, these values cannot be modified without invalidating the proof. This guarantee does not establish the real-world authority of the issuer or the truthfulness of issuance. Ontology and requester-allowlist predicates likewise establish membership only relative to provider-committed sets. Issuer-key governance, credential revocation, real-world identity verification, and protection against a compromised TPS remain external trust and lifecycle functions.

4.2. Public-Chain Adversary

The primary adversary is a passive public-chain observer. The adversary can inspect smart-contract code and monitor public blockchain artifacts across multiple authorization sessions, including policy commitments, submitted proofs, public inputs, emitted events, calldata, timing, and gas usage. The adversary does not observe provider-to-TPS policy material, commitment-opening randomness, requester-submitted authorization evidence, credential witnesses, or TPS-side witness construction: these items are exchanged over off-chain channels or held locally by the provider and the TPS, and never appear in on-chain artifacts. We assume the off-chain channels themselves are confidential (e.g., TLS-protected).

Each policy-enforcement instance corresponds to an authorization session, in which a requester seeks access to a protected resource under a committed policy. For each such session, the adversary observes a public view

$$view_{\mathsf{pub}} = (h, \pi, id_{\mathrm{req}}, id_{\mathrm{res}}, nonce, pp, meta), \tag{1}$$

where h is the on-chain commitment to the hidden policy, π is the zero-knowledge proof submitted for on-chain verification, id_{req}, id_{res}, and *nonce* are the public requester, resource, and session-binding values, *pp* denotes other fixed-format public parameters or public handles, and *meta* denotes transaction-level metadata exposed by the enforcement layer, such as calldata footprint, event structure, public-input layout, timing, and gas usage. The adversary's goal is to infer hidden information about the committed policy or the authorization interaction from this public view.

In addition to the public-chain observer, we treat the requester as untrusted with respect to policy confidentiality. The requester initiates access requests, observes its own submitted evidence and authorization outcome, and inspects the public transcript. However, the requester does not receive the encoded policy, commitment-opening randomness, or policy-bound proving material, which remain within the provider-to-TPS proof-generation boundary. This restriction matters because a requester that learned the policy could infer sensitive facts about the data subject from the consent terms themselves and, in addition, would learn the policy's full acceptance region—which attribute combinations are granted—enabling targeted evidence construction or probing beyond its own legitimate sessions.

Adaptive requester probing and off-chain side channels are outside the scope of the present evaluation. The proved authorization outcome is bound to a pseudonymous requester identifier, a protected resource, and a session nonce through the context digest defined in Section 5. This binding prevents substitution of the recorded authorization context, but it should not be interpreted as proof of the requester's real-world identity.

4.3. Privacy Objectives

We operationalize policy privacy as three measurable, attack-oriented objectives that cover the information a public-chain observer could extract about a hidden policy: its semantic content, its identity relative to other policies, and its structural form. These correspond to Policy Content Hiding, Policy Indistinguishability, and Structure and Parameter Privacy, respectively.

Policy Content Hiding (PCH). PCH requires that public-chain artifacts should not reveal the semantic content of the hidden consent policy, such as data-use terms, disease categories, research purposes, institutional restrictions, or ethics-related requirements.

Policy Indistinguishability (PI). PI requires that different candidate policies should not become distinguishable from their public verification artifacts when they are compiled into the same fixed public interface. This captures whether two policies can be told apart from commitments, proofs, public parameters, or metadata even when their direct representations are hidden—that is, whether the identity of the governing policy can be narrowed down.

Structure and Parameter Privacy (SPP). SPP requires that public-chain artifacts should not reveal structural or parametric properties of the hidden policy, such as logical structure, nesting depth, predicate count, set size, or threshold-like parameters.

4.4. Problem Statement

Given a provider-defined consent policy P, requester authorization evidence, a globally observable public blockchain, and the stated TPS trust assumption, the problem is

to enforce access to a protected healthcare resource while satisfying three requirements. First, authorization must be publicly verifiable: the ACM should be able to verify that the requester satisfies the committed policy without learning or executing the policy in plaintext. Second, policy-dependent information should remain hidden from public-chain observers, including policy content, policy identity, structural properties, requester evidence, and metadata-level enforcement fingerprints. Third, authorization outcomes should remain auditable, so that successful access decisions can be recorded and checked without exposing the hidden policy logic.

5. TPS-ZKPI Design

5.1. Overview

As shown in Figure 1, the TPS-ZKPI workflow binds a provider-committed hidden policy to requester-submitted authorization evidence through TPS-side proof generation and on-chain proof verification. The provider registers only a policy commitment on-chain and sends policy-bound proving material to the TPS off-chain (steps 1–2 in the figure). For each access request, the TPS constructs the private witness from the hidden policy material and requester evidence, generates a zero-knowledge proof of policy compliance, and submits a fixed-format proof statement for ACM verification (steps 3–4). The ACM records the authorization outcome in contract storage and emits it in the blockchain event log (as access-granted or access-denied events; step 5); the data access layer consults the recorded decision to serve or refuse the request (steps 6–9).

This design separates confidentiality-sensitive proof generation from public verification. Policy material and requester evidence remain off-chain—policy material within the provider–TPS boundary, requester evidence between the requester and the TPS—while the blockchain verifies only a fixed-format proof statement against the provider's policy commitment, alongside session parameters validated by the ACM. In a traditional requester-as-prover design, the requester generates the proof itself and must therefore receive the encoded policy and the commitment-opening randomness, so every requester learns the policy in normal operation. In TPS-ZKPI, this material never leaves the provider–TPS boundary: the requester receives no policy material, and policy confidentiality can fail only if the TPS trust assumption itself is violated.

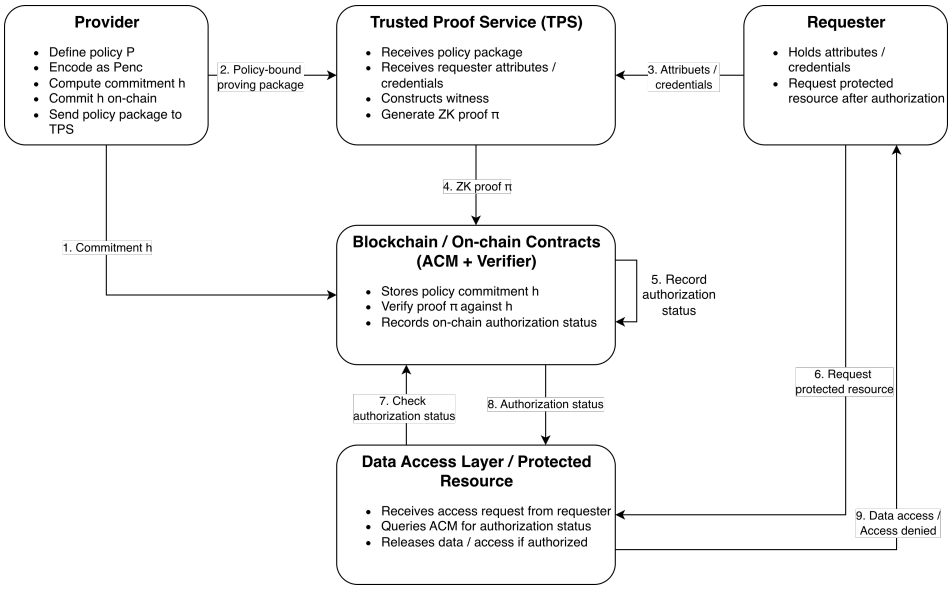


Figure 1. TPS-assisted policy-compliance workflow (steps are described in the text).

5.2. Policy Representation and Interpretation

TPS-ZKPI represents heterogeneous Boolean-based consent policies in a fixed-length, circuit-friendly format. This policy class is chosen for two reasons. First, the targeted biomedical consent conditions—DUO-style data-use terms, disease-scope constraints, and institutional or ethics requirements—are categorical conditions combined with AND/OR/NOT, and conditions that are not natively Boolean enter through fixed external-predicate slots. Second, a Boolean policy class admits a fixed-length encoding evaluated by a single reusable interpreter circuit, which keeps the verification interface policy-independent—a prerequisite for the privacy objectives evaluated later. Let

$$X = \{0,1\}^N, \qquad B = \{0,1\}^M \tag{2}$$

denote the requester-attribute assignment space and the external-predicate assignment space, respectively. The full assignment space is

$$O = X \times B, \tag{3}$$

where $a \in X$ encodes requester attributes and $b \in B$ encodes predicate outcomes, including credential, ontology-membership, requester-allowlist, and obligation-related checks. The implemented relation derives these bits from their associated witnesses rather than accepting unconstrained predicate assertions. The strength of the resulting provenance guarantee nevertheless depends on the mechanism: the profile-credential slot verifies an issuer signature in-circuit, whereas ontology and requester-allowlist slots establish membership relative to provider-committed sets. Issuer authorization, issuance practice, and credential revocation remain external lifecycle functions (Section 4.1).

Ontology-linked conditions are represented extensionally rather than through in-circuit ontology reasoning. At encoding time, the provider expands an ontology concept into a whitelist of codes and commits the whitelist as a Merkle root among the policy's per-predicate parameters; in-circuit, the predicate check is a set-membership proof of the requested term against this root. A disease-specific policy scoped to diabetes, for example, whitelists the ICD-10 diabetes subtree, whereas one scoped to type-1 diabetes whitelists only the type-1 codes. Resolving subsumption at encoding time keeps ontology reasoning out of the circuit and preserves the policy-independent interface.

Consent policies are expressed as Boolean formulas over assignment bits:

$$P ::= atom \mid (P \wedge P) \mid (P \vee P) \mid \neg P, \qquad atom ::= x_i \mid y_j. \tag{4}$$

Here x_i selects requester-attribute bit a_i and y_j selects external-predicate bit b_j. For example, the consent condition "the resource may be used for disease-specific research on diabetes, or for clinical care under approved clinical terms" is expressed as $(x_1 \wedge y_1) \vee (x_2 \wedge y_2)$, where x_1 and x_2 flag the disease-specific-research and clinical-care purposes among the requester attributes, y_1 attests that the requested disease code lies in the committed diabetes whitelist, and y_2 attests that the requested clinical term lies in the approved clinical whitelist. Each policy is compiled into a Reverse Polish Notation (RPN) program over operand tokens and Boolean operator tokens. RPN linearizes arbitrarily nested formulas into a parenthesis-free token sequence that a stack machine evaluates in a single left-to-right pass with a fixed step count—a control structure that maps directly onto a fixed-size circuit and lets one interpreter serve all policies. To support reusable verification, each program is padded to a fixed token length L:

$$P_{\text{enc}} = [t_1, \ldots, t_L], \tag{5}$$

together with an explicit program length $\ell \leq L$ carried as a private witness. The interpreter evaluates only the first ℓ tokens; padding positions beyond ℓ are inactive and leave the output unchanged. The example policy above compiles to the token sequence $[x_1, y_1, \wedge, x_2, y_2, \wedge, \vee]$ with $\ell = 7$; the remaining $L - 7$ positions are padding. The implemented RC1 circuit uses $N = 13$, $M = 11$, and $L = \mathsf{DEPTH} = 64$. We denote the set of length-L programs by Prog_L.

The reusable policy interpreter is a zero-knowledge circuit

$$C : \mathsf{Prog}_L \times \{0, \ldots, L\} \times O \to \{0, 1\}. \tag{6}$$

Given an encoded policy, its program length, and operand values, the interpreter evaluates the RPN program using deterministic stack semantics and enforces well-formedness constraints inside the circuit. The circuit outputs 1 only if the requester evidence satisfies the encoded policy. Well-formedness is enforced as hard constraints: for a program that violates the stack discipline, no satisfying witness exists and no proof can be generated. An output of 0 therefore indicates unsatisfied evidence for a well-formed program. The proved acceptance relation is the asserted case $C(P_{\text{enc}}, \ell, (a, b)) = 1$.

The fixed-length interpreter thus realizes the policy-independent interface motivated above, and supports policy evolution through different fixed-length policy programs.

Scope of the supported policies. The policy class is deliberately restricted, and Table 4 states what the RC1 prototype establishes end-to-end. Categorical data-use terms, ontology-scoped conditions, requester allowlists, profile-backed terms, the policy validity window, and time-limited use are decided at authorization time, either inside the proved relation or by the ACM from public state. A publication moratorium is bound into the policy as an accepted obligation but cannot be decided at request time, negation is rejected by the DUO compiler because membership witnesses prove inclusion only, and purpose limitation, post-access duties, delegation, and credential revocation are left to the data access agreement and the issuer lifecycle. The prototype is therefore an access-time consent checker for bounded Boolean policies over standardized terms, not a general consent-language engine.

5.3. Commitment Binding and Proof Generation

To bind each proof to a provider-defined hidden policy, the provider samples commitment randomness r uniformly from the scalar field and computes

$$\begin{aligned} q &= \mathsf{H}_{\text{pol}}(P_{\text{enc}}), \\ h &= \mathsf{Com}(P_{\text{enc}}, \ell; r) = \mathsf{Poseidon}(q \parallel \ell \parallel r), \end{aligned} \tag{7}$$

where q is the Poseidon-based digest of the fixed-length encoded policy, ℓ is the private program length, r is the commitment randomness, and h is the policy commitment stored on-chain. The semantics of an encoded policy are determined by the pair (P_{enc}, ℓ), since ℓ identifies the active prefix of the padded token array. Accordingly, the circuit supplies the same private value ℓ to both the commitment verifier and the policy interpreter. Under the collision resistance of Poseidon, an accepting proof bound to h cannot substitute a different program length and therefore cannot evaluate an alternative stack-consistent prefix of P_{enc}. Hiding is an explicit commitment assumption (Section 4.1) that relies on high-entropy r and the fixed-length encoding of P_{enc}, which keeps the sponge input unambiguous.

For an access request, let

$$\chi = (id_{\text{req}}, id_{\text{res}}, \nu), \qquad d_\chi = \mathsf{Poseidon}(id_{\text{req}} \parallel id_{\text{res}} \parallel \nu), \tag{8}$$

Table 4. Consent-policy features and how far the RC1 prototype establishes them. "In-circuit" means the condition is part of the proved relation; "on-chain" means the ACM enforces it from public state; "committed" means the value is bound into the policy but compliance is not decided at request time.

Consent feature	Status	What is established at authorization time
Categorical data-use terms combined with AND/OR (13 non-procedural DUO attributes, programs up to 64 tokens)	In-circuit	The committed program and its length are evaluated by the interpreter; the outcome is the proof's output.
Disease, study-design, ancestry, geography, and clinical-term scopes (DS, RS, POA, GS, CC)	In-circuit	Merkle membership of the requested code in a provider-committed whitelist whose root is bound in the parameter commitment.
User-, project-, and institution-specific restrictions (US, PS, IS)	In-circuit	Membership of the holder pseudonym in a provider-committed allowlist for the scope.
Profile-backed terms (GRU, HMB, NMDS, GSO, NPUNCU, NPOA, NPU, NCU)	In-circuit	EdDSA-Poseidon credential over policy commitment, pseudonym, resource, claims, validity interval, and identifier; the attributes are derived from the signed claims.
Policy validity window	In-circuit + on-chain	The committed window is checked against the public request epoch, which the ACM requires to equal the current hour.
Time-limited use (TS)	On-chain	The committed duration starts a per-requester window on the first grant; later requests outside it are denied.
Publication moratorium (MOR)	Committed	The deadline is bound into the policy as an accepted obligation; publication conduct is not observable at request time.
Negation	Not exposed	The interpreter evaluates NOT, but the DUO V3 compiler rejects it: membership witnesses prove inclusion only.
Consent withdrawal and policy evolution	On-chain	A successor policy supersedes its predecessor, a policy can be revoked, and a resource can be rebound; earlier grants remain auditable but are no longer current.
Purpose limitation, post-access duties (PUB, COL, RTN), delegation, credential revocation	Not provided	Carried by the data access agreement and issuer lifecycle, outside the proved statement.

where ν is a fresh session nonce. Here id_{req} is a pseudonymous authorization identifier. For policies that activate a holder-bound credential or requester-allowlist predicate, the circuit derives it as $\mathsf{Poseidon}(s_{\text{holder}})$ from a private holder secret s_{holder}; otherwise, it may be a requester-supplied pseudonym. The TPS constructs the private circuit witness

$$w = (P_{\text{enc}}, \ell, a, b, r, \omega, \chi), \tag{9}$$

where P_{enc} is the encoded policy, ℓ is its program length, a denotes requester attributes, b denotes external predicate outcomes, r is the commitment randomness, and ω denotes auxiliary witnesses for credential- or predicate-dependent checks. The context tuple χ consists of private circuit inputs, while its digest d_χ is public. The current public-signal vector is

$$(1, h, pp, d_\chi, \tau), \tag{10}$$

where pp is a fixed-format commitment to the per-predicate parameters and τ is the request epoch used by the temporal checks. Although the three components of χ are not separate Groth16 public inputs, the ACM receives them as transaction parameters in order to recompute d_χ; they are therefore visible in transaction calldata.

For the compiled circuit, the Groth16 [23] setup produces the common reference string as a proving key and a verification key, once for the shared circuit:

$$(pk, vk) \leftarrow \mathsf{Setup}(C). \tag{11}$$

The TPS generates a proof

$$\pi \leftarrow \mathsf{Prove}\big(pk, w; (1, h, pp, d_\chi, \tau)\big). \tag{12}$$

The proof certifies the conjunction that $C(P_{\text{enc}}, \ell, (a, b)) = 1$, $h = \mathsf{Com}(P_{\text{enc}}, \ell; r)$, $d_\chi = \mathsf{Poseidon}(\chi)$, and the selected external-predicate witnesses satisfy their policy-bound verification relations. Before verifying π, the ACM checks that the resource is governed by the stated policy, that τ is current, that the nonce has not been used for the same requester–resource context, and that the digest recomputed from the transaction parameters equals the public signal d_χ. Thus, within one ACM deployment, a proof generated for one requester identifier, resource, and nonce cannot be accepted under a different tuple. The ACM then verifies π against vk and records the outcome of each request: a positive authorization outcome requires both a valid proof and an asserted output of 1; otherwise the request is recorded as a denial. The policy encoding, commitment randomness, requester attributes, credential witnesses, and auxiliary proof material are not posted on-chain.

5.4. Credential-Origin Binding

The circuit reserves one fixed predicate slot for a credential carrying a fixed set of DUO-relevant claims. Let $c_0, \ldots, c_6$ be the private, issuer-signed claim bits for, respectively, research use, health or medical use, methods development, genetic study, ancestry, commercial use, and non-profit use. Let t_{from} and t_{until} define its validity interval, and let γ be its credential identifier. Let d_{DUO} be the fixed credential schema/domain tag for this credential type. From these signed claims, the circuit derives the DUO terms GRU, HMB, NMDS, GSO, NPUNCU, NPOA, NPU, and NCU. For issuer public key $A = (A_x, A_y)$, the circuit reconstructs the domain-separated message

$$m_{\text{cred}} = \mathsf{Poseidon}(d_{\text{DUO}} \parallel h \parallel id_{\text{req}} \parallel id_{\text{res}} \parallel c_0 \parallel \cdots \parallel c_6 \parallel t_{\text{from}} \parallel t_{\text{until}} \parallel \gamma) \tag{13}$$

Before proof generation, an issuer holding the corresponding private signing key s_{iss} issues the credential by computing

$$\sigma \leftarrow \mathsf{Sign}_{\text{EdDSA-Poseidon}}(s_{\text{iss}}, m_{\text{cred}}). \tag{14}$$

The resulting credential is supplied to the TPS as private off-chain evidence for proof generation. At proof generation, the TPS supplies A, σ, and the credential fields as private witness values. The circuit then enforces

$$\mathsf{Verify}_{\text{EdDSA-Poseidon}}(A, \sigma, m_{\text{cred}}) = 1, \qquad \mathsf{Poseidon}(A_x \parallel A_y) = k_{\text{iss}}, \tag{15}$$

where k_{iss} is the issuer-key commitment contained in the policy-bound parameter commitment pp. The circuit additionally requires $t_{\text{from}} \leq \tau \leq t_{\text{until}}$ and $\gamma \neq 0$. Because this credential is holder-bound, a policy that activates this slot also requires $id_{\text{req}} = \mathsf{Poseidon}(s_{\text{holder}})$, binding the credential to the pseudonymous requester identifier in the authorization context. The corresponding DUO attributes are constrained to these claim-derived values.

5.5. Transcript Normalization

Zero-knowledge proofs hide private witness values, but surrounding blockchain artifacts may still leak policy-dependent information. TPS-ZKPI therefore normalizes the public transcript so that different policies induce fixed or tightly bounded observable profiles. Table 5 summarizes the mechanisms.

Table 5. Transcript-normalization mechanisms in TPS-ZKPI.

Artifact	Normalization mechanism
Calldata	Fixed verification interface and fixed-size proof/public-input encoding.
Logs	Fixed event structure with constant topic and payload layout.
Public parameters	Fixed-format handles instead of policy artifacts or credential witnesses.
Gas usage	Policy-independent verifier logic and randomized gas padding toward fixed targets.

These mechanisms do not eliminate all public observability. Rather, they reduce policy-dependent variation in calldata, logs, public parameters, and gas-related behavior that could otherwise serve as a fingerprint for the metadata-only adversary defined in Section 4. TPS-ZKPI does not flatten all residual channels: timing between transactions, transaction ordering, mempool or broadcast patterns, and off-chain TPS communication remain outside the normalized public transcript.

6. Privacy Evaluation

We evaluate PCH, PI, and SPP (Section 4.3) against a passive public-chain observer (Section 6.1). The evidence comes from two attack families run on the same executions: a supervised attacker that tries to recover each hidden label from the on-chain observables, and an unsupervised attacker that asks whether the observables cluster along that label without seeing it. Section 6.2 defines both pipelines and the multi-seed criterion that decides each privacy verdict. Across the 45 tasks, two meet the failure criterion (one ICD-10 chapter and one nesting-depth contrast); both trace to the random pad target rather than to the policy, and the unsupervised attacker finds no natural grouping by the hidden labels beyond the same coincidence. Beyond the privacy verdicts, we report the system's performance as a cost profile: per-transaction gas for commit and access (including the padding overhead), calldata sizes, circuit constraint count, and proving time, together with how the prover-side cost scales with the program length (Section 6.9, Tables 15 and 16).

6.1. Adversary and observables

We instantiate the passive public-chain observer of Section 4: an adversary who reads the ledger without restriction and sees every transaction and event our approach emits, but never the plaintext policy or the witness.

Under the assumptions of Section 4.1, the byte values in the adversary's public view carry no policy information: the proof is zero-knowledge, the two commitments in the public statement are hiding, and the authorization output is identical for every execution in the corpus. The attack therefore targets the metadata of these artifacts: for each policy execution (a commit transaction followed by an access transaction) it extracts the following features:

- **Gas:** `commitGasUsed`, `accessGasUsed`.
- **Calldata length:** `commitCalldataLength`, `accessCalldataLength`.
- **Proof size:** `proofSize`.
- **Event logs:** `commitLogsCount`, `commitLogsSize`, `accessLogsCount`, `accessLogsSize`.

Of these features, calldata length, proof size, and log counts/sizes are constant across policies by design; any policy-dependent signal can therefore only flow through the two gas measurements (`commitGasUsed`, `accessGasUsed`). The fixed circuit and the commitment-only interface make the non-gas features constant; randomized gas padding protects the gas channel that remains. That channel is wider than the verifier's arithmetic: because the EVM prices calldata per byte (four gas for a zero byte, sixteen otherwise), the gas of an access transaction also encodes the byte pattern of its public arguments, so any deterministic difference in how an argument is encoded reaches the observer through gas even when the calldata length is constant (Section 6.9 exhibits one such difference). The constant features remain in the attack pipeline but carry no variation to exploit. We do not model side channels such as block timing, transaction ordering, mempool or broadcast patterns, or off-chain TPS communication, nor the non-goals stated in Section 4.

We consider three adversary goals, one per privacy objective: (i) recover the policy's *content* (PCH, Section 6.4); (ii) distinguish two *semantically related* policies (PI, Section 6.5); and (iii) recover the policy's *structure or parameter sizes* (SPP, Section 6.6).

6.2. Attack methodology

We evaluate each privacy objective with a primary supervised attack and a complementary unsupervised cross-check. The *supervised* method asks whether a trained classifier can recover a hidden label from the observables, measured by test AUC against the chance level of 0.5. Its outcome serves as the main privacy verdict. The supervised adversary has strictly more information than the unsupervised one, and is thus the stronger of the two. Because this adversary is harder to defeat, a PASS under the supervised test is the more conservative verdict. The *unsupervised* method asks whether the observables cluster along the hidden label, measured by the Adjusted Rand Index (ARI) and Normalized Mutual Information (NMI) against their chance level of 0. This chance level is exact for ARI but only nominal for NMI, which is positively biased at small samples (Section 6.8).

Execution setup. All policy executions are transactions on the *Polygon Amoy* testnet: for each generated policy, we commit the encoded policy on-chain from a provider account (`commitPolicy`), generate the Groth16 proof locally in the role of the provider-operated TPS (Section 4.1), and submit the access transaction from a separate TPS account (`requestAccessBound`). The provider-to-TPS and requester-to-TPS exchanges are in-process calls: they use no public channel and produce no on-chain artifacts. They therefore do not affect the observables of Section 6.1, which are extracted from the resulting transaction receipts, calldata, and event logs.

Both methods run on the same dataset of *Polygon Amoy* executions, collected under the full defense configuration (fixed circuit, commitment-only interface, randomized gas padding). The corpus composition and the two-layer execution design that supplies balanced PI pairs are detailed in Section 6.3 and Table 6. Per-task sample sizes are listed in the result tables. The dataset, attack code, and verdict files are released as an accompanying artifact (anonymized for review).

6.2.1. Supervised attacks

Each objective is operationalized as a set of binary *tasks*, each defined by one hidden label to be recovered from the observables: detecting the presence of a specific data-use term or disease chapter (PCH, Section 6.4), distinguishing executions of two policies (PI, Section 6.5), or separating two structural buckets (SPP, Section 6.6). For every task we train a linear and a non-linear classifier (logistic regression, LR; random forest, RF) and report the score that enters the verdict for that objective (defined below). Each task is evaluated over **30 random seeds** using a **policy-disjoint train/test split** (the same 70/30 ratio in each of PCH, PI, and SPP): the policies used to train the attacker never appear in the test set, so a high test AUC reflects leakage that generalizes across policies. Because the split assigns whole policies rather than individual executions, we cannot partition executions to hit the target class proportions exactly. For each seed we therefore approximate stratification by sampling many candidate splits (512 per seed in our runs) and keeping the one whose per-class test proportions are closest to 70/30. Here a class is one side of the binary task (the positive or negative value of its hidden label). By keeping each class's test share close to the target ratio, the candidate search ensures every seed's test set contains both classes, so each seed's AUC is estimated on a two-class test set. For each seed we assess the AUC with a label-permutation test. The classifier is trained once and its test-set probability outputs are fixed; the test-set labels are then shuffled N=500 times, and the AUC is recomputed against the fixed probabilities on each shuffle, giving the null distribution of the AUC under no association between scores and labels. The empirical p-value is $(b+1)/(N+1)$, where b is the number of shuffles whose AUC meets or exceeds the observed (unshuffled) AUC; the added counts keep the p-value away from an exact zero. The seed is counted as significant when this p-value falls below α=0.05.

For each task, the result tables report the AUC averaged over seeds (AUC_{mean}) and the mean per-seed distance from chance, $\text{mean}_s\,|\text{AUC}_s - 0.5|$ (reported as the AUC gap). Because the gap averages per-seed absolute deviations, it is generally larger than $|\text{AUC}_{\text{mean}} - 0.5|$ computed from the mean AUC. This AUC gap is an empirical, sample-estimated diagnostic, not the advantage of a formal cryptographic distinguisher and not a proven security bound. Unlike accuracy, whose chance level shifts with the number of classes, AUC is insensitive to class count and prevalence: a one-vs-rest AUC is a prevalence-weighted average of pairwise AUCs, so the results are not tied to the class counts of our generated corpus. Under any number of circulating classes, the one-vs-rest AUC decomposes into pairwise contrasts of the kind that PI and SPP measure directly; if every pairwise contrast sits at chance, so does any prevalence-weighted average of them. The PASS/FAIL decision, defined below, combines AUC_{mean} with the significant-seed fraction—the fraction of seeds whose per-seed permutation test rejects at α=0.05, reported as %sig in the result tables. The verdict is:

> **FAIL** iff $\text{AUC}_{\text{mean}} > 0.55$ **and** the significant-seed fraction is ≥ 0.5 (over 30 policy-disjoint seeds); **PASS** otherwise.

Both conditions of this *double criterion* are required: a high AUC that does not reproduce across seeds is not a leak, and neither is a reproducibly significant but near-chance AUC.

Choice of thresholds. The two thresholds are application-set rather than derived, and we justify them by what each leg guards against. The AUC threshold of 0.55 asks for an effect at least twice the seed-to-seed standard deviation of the AUC on our task sizes (0.02–0.03 for the DUO tasks, 0.06–0.08 for the smaller ICD-10 tasks), so that a task cannot fail on the sampling noise of a single split; the 50% threshold asks that the permutation test reject in a majority of the 30 policy-disjoint splits, so that a task cannot fail on one lucky partition either. A single leg is not enough: with the AUC leg alone, 4 PCH tasks and 4 SPP contrasts would fail at 0.55 (8 and 12 at 0.52), and with the significance leg alone the same two tasks fail at 50% but three SPP contrasts at 30%. The conclusions are insensitive to the exact values: re-evaluating every task from its recorded per-seed AUCs and p-values over AUC thresholds 0.52–0.60 and significance fractions 0.3–0.7 fails at most the two tasks discussed in Sections 6.4 and 6.6 whenever the significance fraction is at least 0.5, adds at most two further SPP contrasts when it is lowered to 0.3–0.4, and fails nothing once it reaches 0.6. The criterion has a measurable false-positive rate, because the padded gas contains a per-policy random draw and the coverage-layer policies are executed once, so the 30 seeds resample the same draws: replacing the two gas features by policy-independent draws from the padding bands and rerunning the whole pipeline ten times produces at least one spurious PCH failure in four of the ten replicates (a per-task rate of 3%; none on SPP). We therefore treat a failure as a prompt to examine what the classifier separates rather than as proof of leakage, and Sections 6.4 and 6.6 do so by subtracting the logged pad target from the access gas, an explanatory analysis that does not alter the criterion or the verdicts.

Tasks whose test split contains fewer than 20 executions (median over the 30 seeded splits) are *underpowered*—reported but excluded from the verdict, so a small task's sampling noise cannot decide it. The double criterion is applied to LR and RF independently for each task; a task fails if either classifier meets both conditions, and the result tables report the stronger (higher-AUC) classifier for each task.

As established in Section 6.1, the transcript normalization leaves gas as the only policy-dependent channel; the LR/RF feature set is therefore the two gas measurements. A near-chance result alone cannot show whether the defense works or the attacker is simply too weak to find any signal. We therefore pair the main results with the gas-normalization positive control in Section 6.9: the same features, splits, and classifiers recover signal on the un-padded gas transcript, and that signal collapses after padding.

6.2.2. Unsupervised clustering attacks

The clustering attacker drops the labels entirely and asks whether the gas features *naturally* group along the hidden property. We run K-Means (with K set to the number of classes, or K=2 for policy pairs) on the on-chain gas features, and score the recovered clusters against the true labels with ARI and NMI. Values near 0 mean the clusters carry no information about the hidden label; values near 1 mean full leakage.

PCH clusters all executions globally (no per-layer splitting), so the large-sample ARI/NMI are read directly. SPP is clustered the same way, once per annotation axis of Section 6.6 with K equal to that axis's bucket count, and its large-sample ARI/NMI are likewise read directly.

For PI, clustering is scored per pair, so each side of a pair must contribute enough executions; this requires two safeguards. **Balanced pairs.** The clustering uses a dedicated pair set built from the same executions: the policies with ≥ 30 executions each (the depth layer of Section 6.3) are paired into 127 balanced 30-vs-30 contrasts. The pair set of the supervised PI attack (Section 6.5) is unsuitable for per-pair clustering: most of its pairs hold a single execution per side, and its most imbalanced hold 1 execution against 30, so K-Means cannot recover a meaningful partition, and ARI/NMI ≈ 0 would reflect the

degenerate setup rather than privacy. (The supervised attack itself is unaffected: it pools executions within a pair type rather than partitioning each pair.) **Permutation-debiased NMI.** Even after balancing, each side of a pair holds only 30 executions. At that scale the null simulation in Section 6.8 shows that finite-sample effects alone can raise NMI above zero while ARI stays at chance, so ARI is read directly and NMI must be debiased. The cluster assignments are computed once (K-Means does not see the labels) and the observed NMI is averaged over 20 K-Means seeds; the pair labels are then shuffled 200 times and the NMI recomputed against the fixed assignments on each shuffle, giving the null distribution of NMI under zero true signal. The debiased NMI is the observed NMI minus the mean of this null distribution, and the empirical p-value p_{NMI} is the fraction of shuffles whose NMI meets or exceeds the observed value. The cross-check flags a pair only if its debiased NMI is positive *and* significant ($p_{\mathrm{NMI}} < 0.05$).

6.3. Dataset and corpus generation

Shared corpus. We evaluate all three objectives on one policy corpus, so the supervised attacks and the clustering cross-check see the same transactions. The corpus holds **1077 distinct policies** run as **1773 on-chain executions** on *Polygon Amoy*, each execution being one commit transaction followed by one access transaction. The policies are produced by a deterministic generator with a fixed seed (`unified-decoupled-2026`). For reproducibility, this generator uses a deterministic test issuer to create the private profile-credential witnesses used by policies that exercise the credential predicate; this testing key does not model issuer governance or production key management. For each policy the generator picks its content and its structure. The content is a set of DUO data-use terms drawn from ten terms: five ontology-bound terms, each checked against a code set from a biomedical vocabulary (DS against ICD-10, RS against the Ontology for Biomedical Investigations, OBI [37], POA against HANCESTRO, GS against ISO 3166, CC against SNOMED CT), and five profile-backed terms (GRU, HMB, NMDS, GSO, NCU) that are derived from the signed claims of the requester's profile credential (Section 5). The structure is the logical form joining the terms; whitelist sizes are drawn once per policy from a global set of buckets, independently of the logical form, so that the four structural axes of Section 6.6 are not functions of one another. For example, one generated policy permits research-specific and disease-specific use (DUO terms RS and DS) and grants access only when *both* conditions hold, joined by `AND`: the study design belongs to the OBI *observational and comparative designs* family, and the requested record falls in the ICD-10 *diseases of the circulatory system* chapter. Each condition is one predicate—hence the two predicates—and each is checked against a whitelist of the codes it admits: 8 OBI study-design codes and 8 ICD-10 codes drawn from that chapter, the "medium" parameter-size bucket of Table 6. Every code set is a real subset of its vocabulary (each of the 30 scopes holds at least 32 resolvable identifiers), and the recorded whitelist size is the number of leaves actually committed. Table 6 gives the resulting counts for every label and structural bucket.

Two-layer execution design. We run the policies in two layers that serve different needs. The *coverage layer* runs 1053 policies once each. It provides breadth, spreading distinct policies across the content labels and structural buckets, so the PCH and SPP attacks have enough different policies to train on one set and test on another. The *depth layer* runs 24 policies 30 times each (720 executions). It provides depth, which the PI attacks need because they compare two policies execution-by-execution, and for that each policy must have multiple executions of its own. This layer is what lets PI form its 127 balanced 30-vs-30 pairs and run the per-pair clustering check (Section 6.2).

Label restriction and statistical power. We restrict the generated content labels to those with enough support for a statistically powered attack: ten DUO data-use terms and the

ten ICD-10 chapters, sampled so that the smallest class reaches at least 30 executions. The complete DUO has 23 data-use terms and ICD-10 has tens of thousands of leaf codes; generating and executing enough policies for every leaf term to reach adequate power would make the on-chain experiment infeasible. Category-level labels are also the right granularity for the privacy question: the observer wants to know whether the *category* of permitted use or disease leaks. This choice is conservative. A coarse label has more executions behind it than a rare leaf term, and more data makes the attack *stronger*; so if the attack cannot recover a coarse label, it would fail even more clearly on a fine one. Passing at the category level therefore does not hide a leak that finer labels would expose. The restriction also cannot flatter the defenses, which behave the same whatever content a policy carries: the commitment-only interface and gas padding are content-agnostic by construction (Section 5).

Collection. Executions were collected as live transactions on *Polygon Amoy* under the defense configuration of Section 6.2 (fixed circuit, commitment-only interface, randomized gas padding); observables are extracted from the resulting receipts, calldata, and event logs. All 1773 executions completed with the outcome the generator planned (every request was granted), and the feature file holds all of them. As established in Section 6.1, transcript normalization leaves gas as the only policy-dependent channel, so the residual per-transaction variation the attacker sees is confined to the two gas measurements.

6.4. Policy Content Hiding (PCH)

Property. PCH requires that the public-chain observer cannot recover the *content* of a policy—its DUO data-use terms or the ICD-10 disease codes it references—better than chance.

Scope of the labels. The attack targets the ten DUO data-use terms of the corpus—general research use (GRU), disease-specific research (DS), research-specific restrictions (RS), population origins or ancestry research (POA), geographical restriction (GS), clinical care use (CC), non-commercial use (NCU), health or medical or biomedical research (HMB), no methods development (NMDS), and genetic studies only (GSO)—and disease codes aggregated to the ten ICD-10 chapters. This label restriction, its statistical-power rationale, and why category-level granularity is a conservative choice are detailed in Section 6.3.

Supervised attack. Each DUO term defines a one-vs-rest binary task, on which we run the multi-seed pipeline of Section 6.2. Table 8 reports the per-term verdicts: every DUO task stays below the 0.55 AUC line, with mean AUC between 0.477 and 0.540; significant-seed fractions run up to 23% (GRU), short of the 50% gate, and with the AUC leg unmet none approaches the failure criterion. The same pipeline applied to the ten ICD-10 chapters (one-vs-rest over the ICD-bearing rows only, median test n=127 per chapter; Table 9) passes nine chapters and fails one. Chapters ch1, ch2, and ch4 sit just above the AUC line (mean AUC 0.558–0.570) but reproduce in at most 23% of seeds, so they are boundary cases rather than stable channels. Chapter ch5 (diseases of the nervous system) meets both legs: mean AUC 0.667 with 53% of seeds significant. We report it as a FAIL of the criterion and examine below what the classifier separates.

What the ch5 classifier separates. The two gas features are the only non-constant observables, and the access-gas measurement is itself the sum of two parts: the target that the TPS draws uniformly at random from the padding band for that execution (Section 5) and the gas that the padded call consumes beyond it, which on Amoy is a near-constant 4,661–4,750 gas for all 1773 executions. The observer sees only the sum, but the capture log records every target, so we can ask how much of the separability survives once the target is subtracted. None does. Over all 20 PCH tasks the rank-based AUC of the access gas coincides with the AUC of the target alone to three decimals, and the AUC of the resid-

Table 6. Composition of the shared policy corpus (generator seed `unified-decoupled-2026`). DUO terms are multi-label, so their counts overlap; ICD-10 rows cover only the ICD-bearing policies; each SPP axis partitions the full corpus.

Grouping	Category	Policies	Executions
DUO data-use term (multi-label; counts = policies/executions containing the term)			
	GRU	302	447
	DS	422	770
	RS	412	760
	POA	392	624
	GS	382	556
	CC	392	769
	NCU	206	380
	HMB	331	447
	NMDS	237	469
	GSO	236	352
ICD-10 chapter (ICD-bearing policies only)			
	ch0	44	131
	ch1	49	49
	ch2	33	62
	ch3	55	55
	ch4	41	70
	ch5	34	63
	ch6	48	135
	ch7	48	77
	ch8	33	62
	ch9	37	66
	ICD-bearing subtotal	422	770
SPP structureType			
	trivial	130	130
	and_only	294	526
	or_only	303	535
	and_or_mixed	350	582
SPP nestingDepth			
	flat	130	130
	shallow	210	413
	medium	335	509
	deep	402	721
SPP predicateCombo			
	single	257	344
	few	645	1080
	many	175	349
SPP paramType			
	none	147	234
	small	232	406
	medium	352	584
	large	346	549
Corpus total		**1077**	**1773**

Table 7. Headline supervised results on the Polygon Amoy dataset (30 policy-disjoint seeds; a task fails iff $AUC_{mean} > 0.55$ and at least 50% of seeds are significant).

Property	Hidden target	Headline supervised result
PCH	DUO terms and ICD-10 chapters	19 of 20 content-recovery tasks pass; AUC 0.464–0.570, significant seeds median 3%. ICD ch5 fails (AUC 0.667, 53%); its separability is carried entirely by the random pad target*
PI	Related policy-pair identity	3 gating tasks pass; AUC 0.509–0.549; significant seeds median 7%, max 10%
SPP	Structure, predicate, parameter, and nesting-depth buckets	20 of 21 pairwise contrasts pass; AUC 0.468–0.571, significant seeds median 7%. `nestingDepth` medium/deep fails (AUC 0.563, 53%); carried by the pad target*

* Rerunning the same protocol on the gas minus the logged pad target returns both tasks to PASS (Sections 6.4 and 6.6).

ual (gas minus target) lies between 0.47 and 0.52; for ch5 the gas and target AUCs are both 0.601 and the residual AUC is 0.472. The 34 ch5 policies simply drew targets whose mean lies 33,000 gas above that of the other 388 ICD-bearing policies, a 3.2σ deviation for a uniform draw of that size; because each coverage-layer policy is executed once, the 30 seeds resample the same 34 fixed draws, which is why the deviation reproduces across seeds. Rerunning the of-record protocol unchanged on the residual in place of the access gas gives mean AUC 0.556 with 10% significant seeds (logistic regression) and 0.561 with 17% (random forest), a PASS; rerunning it on the target alone reproduces the FAIL (0.686, 63%). The padded gas therefore carries a policy-independent random draw that happened to correlate with one chapter in this corpus, not information about the chapter; Section 6.2 quantifies how often the criterion produces such a false positive.

Table 8. Policy Content Hiding: per-term supervised attack over 30 policy-disjoint seeds (one-vs-rest tasks; higher-scoring classifier per task). All ten DUO tasks pass.

Term	AUC (mean ± std)	AUC gap (mean)	%sig	med n
DUO GRU	0.540 ± 0.026	0.041	23%	323
DUO DS	0.514 ± 0.029	0.027	3%	323
DUO RS	0.498 ± 0.028	0.022	0%	323
DUO POA	0.477 ± 0.024	0.027	0%	323
DUO GS	0.502 ± 0.027	0.023	0%	323
DUO CC	0.515 ± 0.029	0.025	3%	323
DUO NCU	0.528 ± 0.027	0.031	7%	323
DUO HMB	0.509 ± 0.027	0.024	3%	323
DUO NMDS	0.516 ± 0.030	0.025	3%	323
DUO GSO	0.502 ± 0.031	0.026	3%	323

6.5. Policy Indistinguishability (PI)

Property. PI strengthens PCH: even when two policies are *semantically related*, the observer must not tell which one was executed. We construct policy pairs of three types, defined by semantic similarity: `semantic_equivalent` (near-identical semantics, similarity ≥ 0.999, but a different parameter fingerprint—whitelist size, Merkle depth, or proof type), `weak_equivalent` (≥ 0.95 semantic overlap), and `controlled_difference` (similarity < 0.95, the least similar pairs by construction; reported for reference but excluded from the ver-

Table 9. Policy Content Hiding: per-chapter ICD-10 supervised attack over 30 policy-disjoint seeds (higher-scoring classifier per chapter).

Chapter	AUC (mean ± std)	AUC gap (mean)	%sig	med n	Verdict
ICD ch0	0.472 ± 0.078	0.064	0%	127	PASS
ICD ch1	0.570 ± 0.066	0.075	23%	127	PASS
ICD ch2	0.565 ± 0.071	0.081	10%	127	PASS
ICD ch3	0.540 ± 0.061	0.061	7%	127	PASS
ICD ch4	0.558 ± 0.061	0.070	13%	127	PASS
ICD ch5	0.667 ± 0.069	0.167	53%	127	FAIL
ICD ch6	0.464 ± 0.077	0.065	0%	127	PASS
ICD ch7	0.525 ± 0.064	0.058	3%	127	PASS
ICD ch8	0.478 ± 0.078	0.063	0%	127	PASS
ICD ch9	0.511 ± 0.074	0.062	0%	127	PASS

dict). Semantic similarity here is a fixed heuristic score over policy semantics—the Jaccard overlap [38] of DUO terms combined with agreement of the ICD-10, OBI, and other scope fields—that returns 1 when two policies share an identical normalized logic string. The ≥ 0.999 threshold therefore selects logically identical policies and ≥ 0.95 their close approximations; the exact component weights are given in the released artifact. To build the pairs we score every candidate policy couple by this similarity, assign it to one of the three types above, and sample a fixed budget of 100 pairs per type, or every available pair when fewer exist (71 `weak_equivalent` pairs); each retained pair is an ordered couple (P_0, P_1) that the supervised attack below tries to tell apart.

Supervised attack. Each pair is an ordered couple (P_0, P_1), and the executed policy defines a binary label $b \in \{0, 1\}$; the attacker's task is to predict b from the on-chain features of a single execution, i.e. to decide which policy of the pair produced it. Indistinguishability is thus a two-class problem whose ideal outcome is a guess no better than a coin toss, corresponding to a classifier AUC of 0.5. We train a logistic-regression and a random-forest classifier and, per pair type, report the higher-AUC one, following the convention of Section 6.2. We evaluate this single-execution attacker over 30 policy-disjoint seeds (Table 10). Pooled over all pairs, the AUC is 0.509 ± 0.035 with 3% of seeds significant. Both gating pair types stay below the 0.55 line: `semantic_equivalent` at 0.549 (7% significant seeds) and `weak_equivalent` at 0.544 (10%), so PI passes. `controlled_difference`, the least similar pairs by construction and hence the easiest to separate, is excluded from the verdict and reported for reference; under the full mechanism it sits at 0.544 with 13% of seeds significant, whereas the ablation of Section 6.9 separates the same pair type at AUC 0.71 once padding is removed, which confirms that the pair construction produces a distinguishable positive control.

Table 10. Policy Indistinguishability: single-execution distinguisher over 30 policy-disjoint seeds, by policy-pair type (higher-scoring classifier per type). `controlled_difference` is reported for reference and excluded from the PI verdict. All gating rows pass.

Pair type	Pairs	AUC (mean ± std)	AUC gap (mean)	%sig	med n
overall (all types)	271	0.509 ± 0.035	0.027	3%	162
semantic_equivalent	100	0.549 ± 0.064	0.070	7%	60
weak_equivalent	71	0.544 ± 0.079	0.079	10%	42
controlled_difference [†]	100	0.544 ± 0.064	0.062	13%	60

[†] Reported for reference; excluded from the PI verdict.

6.6. Structure & Parameter Privacy (SPP)

Property. SPP requires that neither the *shape* of a policy (logical structure, nesting depth, predicate coverage) nor the *size* of its parameters (whitelist/set sizes) leaks to the observer. The supervised attack organizes the test along four annotation axes—logical structure type (`structureType`), predicate count (`predicateCombo`), parameter size (`paramType`), and nesting depth (`nestingDepth`)—and attacks each pair of buckets within an axis. `structureType` has four buckets: `trivial` (a single predicate with no boolean connective, e.g. policy logic `"DS"`), `and_only`, `or_only`, and `and_or_mixed`. `predicateCombo` buckets policies by predicate-namespace count into `single` (1), `few` (2–3), and `many` ($\geq$ 4); `paramType` buckets by whitelist size into `none` (no whitelist: the policy is decided by profile-backed terms only), `small` (1–4), `medium` (5–16), and `large` ($\geq$ 17); `nestingDepth` buckets by the maximum nesting of the normalized expression into `flat` ($\leq$ 1), `shallow` (2), `medium` (3), and `deep` ($\geq$ 4). The generator draws the whitelist size independently of the logical form, so the four axes are not functions of one another (Section 6.3); the per-bucket policy and execution counts are listed in Table 6. Contrasting every within-axis bucket pair gives $6 + 3 + 6 + 6 = 21$ contrasts.

Supervised attack. Table 11 reports all 21 pairwise contrasts over 30 policy-disjoint seeds, with the per-contrast AUC taken from the higher-scoring of the two classifiers following the convention of Section 6.2. Most contrasts sit at chance, with mean AUC between 0.47 and 0.54. Three exceed the 0.55 line without reproducing: `paramType` `none/small` (0.571, 40% significant seeds), `none/large` (0.568, 33%), and `nestingDepth` `flat/medium` (0.555, 27%); the two `none` contrasts separate the profile-only policies, whose 147 policies form the smallest minority class of the axis. One contrast, `nestingDepth` `medium/deep`, meets both legs of the criterion (mean AUC 0.563, 53% significant seeds) and is reported as FAIL. Its separability has the same origin as the PCH ch5 case (Section 6.4). The access-gas AUC of the contrast equals the AUC of the randomized pad target alone (0.440 for both, i.e. deep policies drew lower targets) and the residual after subtracting the target scores 0.467; over all 21 contrasts the gas and target AUCs agree to three decimals, and the residual AUC stays within 0.47–0.56. Rerunning the of-record protocol with the residual in place of the access gas returns the contrast to PASS (logistic regression 0.530 with 13% significant seeds; random forest 0.544 with 33%), while the target alone reproduces the FAIL (0.568, 57%). Reporting AUC rather than an accuracy-style score already discounts majority-class prediction (Section 6.2); what remains above chance on this contrast is the per-policy random draw that the padding itself introduces, not a stable structural channel. Section 6.9 provides the corresponding positive control: when gas padding is removed, the same SPP attacker separates structural and parameter-size buckets with AUCs up to 0.90, and that signal collapses under the full normalization.

6.7. Unsupervised clustering: a label-free cross-check

The supervised attacks (Sections 6.4–6.6) assume an adversary who can train on labeled executions. A weaker but assumption-free attacker simply asks whether the on-chain gas features *cluster* along the hidden property with no supervision at all. Table 12 consolidates this attack across all three objectives: K-Means recovers essentially nothing, with ARI within ± 0.003 and NMI below 0.03 for every DUO, PI, and SPP target except one. NMI values are interpreted against the null finite-sample simulation in Section 6.8, which shows that the slightly higher ICD-10 NMI is consistent with small-sample inflation rather than recoverable signal. The exception is the SPP `nestingDepth` axis, whose ARI of 0.007 is tiny in magnitude but permutation-significant ($p_{\mathrm{ARI}} = 0.005$). It is the unsupervised face of the `medium/deep` contrast of Section 6.6: clustering the same executions on the residual gas (access gas minus the logged pad target) returns the axis to ARI -0.0004

Table 11. Structure & Parameter Privacy: 21 pairwise contrasts across four annotation axes over 30 policy-disjoint seeds (higher-scoring classifier per contrast).

Contrast	AUC (mean ± std)	AUC gap (mean)	%sig	med n	Verdict
structureType					
trivial / or_only	0.520 ± 0.048	0.043	0%	130	PASS
trivial / and_only	0.502 ± 0.040	0.033	3%	127	PASS
trivial / and_or_mixed	0.502 ± 0.042	0.034	0%	144	PASS
or_only / and_only	0.506 ± 0.034	0.027	0%	179	PASS
or_only / and_or_mixed	0.518 ± 0.035	0.032	10%	196	PASS
and_only / and_or_mixed	0.503 ± 0.040	0.032	7%	193	PASS
predicateCombo					
single / few	0.523 ± 0.035	0.033	13%	271	PASS
single / many	0.519 ± 0.048	0.041	7%	130	PASS
few / many	0.527 ± 0.048	0.045	17%	246	PASS
paramType					
none / small	0.571 ± 0.046	0.071	40%	114	PASS
none / medium	0.525 ± 0.035	0.035	3%	150	PASS
none / large	0.568 ± 0.038	0.069	33%	148	PASS
small / medium	0.530 ± 0.037	0.037	20%	175	PASS
small / large	0.504 ± 0.036	0.028	3%	173	PASS
medium / large	0.493 ± 0.027	0.022	0%	209	PASS
nestingDepth					
flat / shallow	0.522 ± 0.038	0.036	3%	102	PASS
flat / medium	0.555 ± 0.032	0.055	27%	140	PASS
flat / deep	0.468 ± 0.046	0.047	3%	160	PASS
shallow / medium	0.521 ± 0.041	0.039	7%	164	PASS
shallow / deep	0.533 ± 0.036	0.039	17%	184	PASS
medium / deep	0.563 ± 0.028	0.063	53%	221	FAIL

($p = 0.60$), while every other axis is unchanged, so the grouping follows the per-policy random draws of the padding rather than the nesting depth. With that qualification, the unsupervised attacker agrees with the supervised verdict as a second, label-free attack family on the same dataset (Figure 2).

Removing sample-size confounds. The PI clustering inputs are rebalanced into 30-vs-30 contrasts and permutation-debiased as described in Section 6.2, so the near-zero ARI/NMI is not a degenerate-measurement artifact: over the 127 balanced pairs the mean ARI is 0.002, the mean debiased NMI 0.001, and 6 pairs (5%) reach nominal significance, the fraction expected under the null at $\alpha = 0.05$. The conclusion matches the supervised PI verdict (Section 6.5), and the ablation (Section 6.9) confirms the clustering device itself is live: on the un-padded transcript the same balanced pairs cluster with a mean debiased NMI 0.67 higher, and the signal disappears after normalization.

Table 12. Unsupervised clustering attack (K-Means, K = number of classes, K=2 for pairs) on the on-chain gas features, with the permutation p-value of ARI. PCH clusters all executions globally.

Property	Target	n	#cls.	ARI	p_{ARI}	NMI
PCH	DUO terms	1773	10	0.0000	0.48	0.0109
PCH	ICD-10 chapters	770	10	0.0021	0.06	0.0281
PI	policy pairs (30v30)	127	2	0.0020	—	0.0139
SPP	`structureType`	1125	4	0.0004	0.33	0.0034
SPP	`predicateCombo`	1125	3	−0.0005	0.58	0.0015
SPP	`paramType`	1125	4	−0.0000	0.49	0.0030
SPP	`nestingDepth`	1125	4	0.0071	0.005	0.0088

6.8. Sample-size sensitivity of NMI

The clustering cross-check (Section 6.7) treats ARI as the primary conservative indicator because NMI can take moderately high values under small samples. To justify that choice, we quantify how *finite-sample effects* alone can inflate NMI even when there is *no*

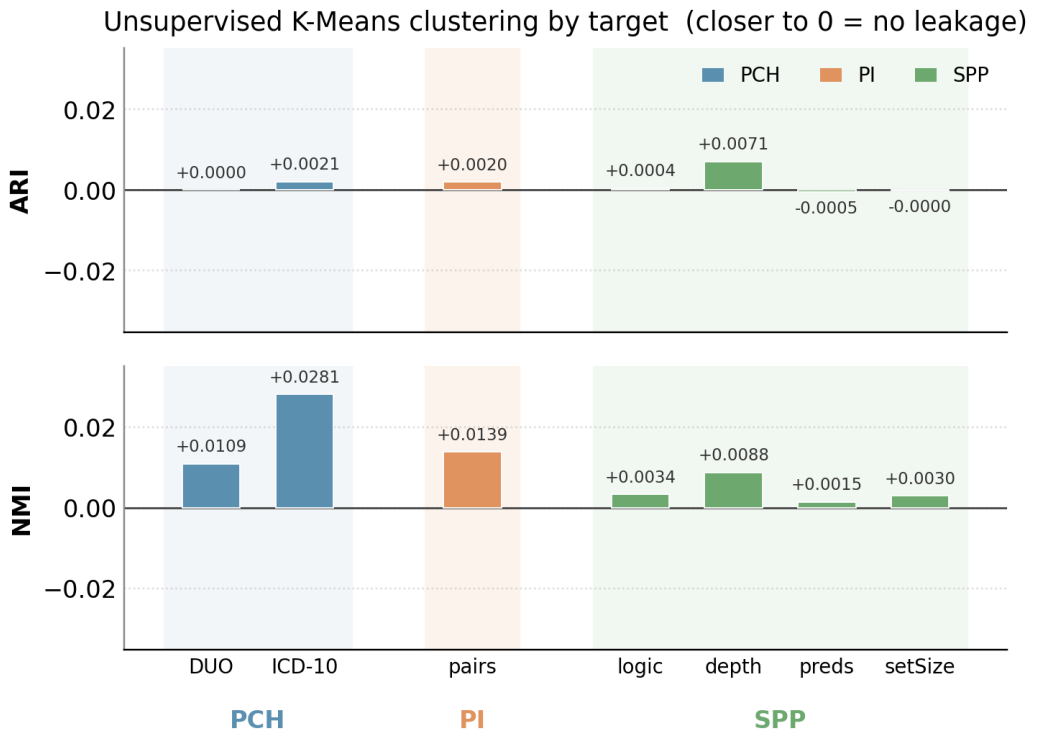


Figure 2. Unsupervised K-Means clustering across all three objectives: ARI (top) and NMI (bottom) per target.

discriminative signal, using an idealized null in which all classes are statistically indistinguishable.

Null hypothesis. We sample every class from the *same* distribution, $x_i \sim \mathcal{N}(0, I_d)$ for all classes, where d is the synthetic feature dimension and I_d is the d-dimensional identity covariance. Since no class carries class-specific information, a clustering algorithm should not recover class membership, and both NMI and ARI should concentrate around 0 as the sample size grows.

Simulation. For each configuration we run $T = 1000$ independent trials. In each trial we (i) generate C classes with n i.i.d. samples per class from $\mathcal{N}(0, I_d)$, assigning each sample a class label by construction order (these labels are independent of the sample values, since all classes share one distribution), (ii) apply K-Means with $K = C$ to the pooled data, and (iii) score the predicted assignments against these assigned class labels with NMI and ARI. We sweep n while holding C fixed, and report the trial means $\overline{\text{NMI}}$ and $\overline{\text{ARI}}$. We consider a **PI-like** binary structure ($C = 2$, $n \in \{3, 10, 50, 100\}$) and an **ICD10-like** multi-class structure ($C = 5$, $n \in \{3, 10, 100\}$).

Results. Table 13 shows that, although all classes are indistinguishable by construction, **NMI is noticeably positive at very small n but collapses toward zero as n grows**, whereas **ARI stays near zero throughout**. For the binary structure, $\overline{\text{NMI}}$ falls from 0.1808 at n=3 (N=6) to 0.0404 at n=10, and to 0.0066/0.0038 at n=50/100, while $\overline{\text{ARI}}$ stays at -0.0190, -0.0020, -0.0011, 0.0002. For the multi-class structure the same trend holds: $\overline{\text{NMI}}$ goes from 0.4168 (n=3) to 0.1249 (n=10) to 0.0105 (n=100), with $\overline{\text{ARI}}$ near zero (0.0005, 0.0018, -0.0000). Hence moderately high NMI in small-sample clustering should not be read as actionable leakage, and ARI is the more conservative indicator of practical recoverability in our unsupervised evaluation.

Table 13. Sample-size sensitivity of NMI and ARI under the null hypothesis where all classes are i.i.d. sampled from $\mathcal{N}(0, I_d)$ (T=1000 trials per row).

Structure	n (per class)	C	N	$\overline{\text{NMI}}$	$\overline{\text{ARI}}$
PI-like	3	2	6	0.1808	−0.0190
PI-like	10	2	20	0.0404	−0.0020
PI-like	50	2	100	0.0066	−0.0011
PI-like	100	2	200	0.0038	0.0002
ICD10-like	3	5	15	0.4168	0.0005
ICD10-like	10	5	50	0.1249	0.0018
ICD10-like	100	5	500	0.0105	−0.0000

6.9. Ablation: gas-normalization positive control

The main results show that the full mechanism hides content, policy identity, and structure/parameters up to the two pad-target coincidences discussed above. The ablation asks *why*: how much of the residual attack surface is removed by randomized gas padding after the fixed circuit and commitment-only interface have already eliminated calldata-length, proof-size, and event-log channels.

We compare two aligned protection profiles on the same selected policy set. **B1** uses the fixed circuit, commitment-only interface, and TPS execution profile without commit/access gas padding. **B2** is the full mechanism used in the main evaluation, with randomized gas padding. The subset is chosen deterministically from the of-record corpus so that every content label and structural bucket keeps at least 70 executions and 8 independent policies and every depth-layer policy is retained: 1119 executions of 423 policies. B1 replays exactly these policies (same tokens, randomizers, and commitments) without padding, so B1 and B2 differ only in the padding calls; B1 was executed three days after B2, on a fresh contract deployment, which the paired analysis treats as a non-contemporaneous control. This is the positive control for the gas channel: calldata length, proof size, and event-log shape are already fixed, so B1 is an un-padded gas-transcript control rather than a run with every normalization layer removed. The paired leakage metric is

$$\Delta_{\text{leak}} = |\text{AUC}_{\text{B1}} - 0.5| - |\text{AUC}_{\text{B2}} - 0.5|, \tag{16}$$

where positive values mean padding moved the attacker closer to chance; we bound each Δ_{leak} with a 95% stability interval over 30 paired train/test splits. Table 14 is intentionally separated from the main privacy verdicts: the main verdicts answer whether the full mechanism is private; this ablation attributes that privacy to a specific defense layer and reports its gas cost.

Padding removes otherwise-recoverable signal. Under the criterion of Section 6.2, the un-padded transcript fails 9 of the 10 DUO tasks, 14 of the 21 SPP contrasts, and the PI positive control, with mean AUCs up to 0.90; the padded transcript of the same subset passes every task. Figure 3 reports the full set of 45 supervised contrasts: each point places a contrast at its unpadded deviation from chance $|\text{AUC}_{\text{B1}} - 0.5|$ against its padded deviation $|\text{AUC}_{\text{B2}} - 0.5|$, so points below the diagonal are contrasts where padding moved the attacker back toward chance. Every one of the 21 contrasts on which B1 carries a sizable signal ($|\text{AUC}_{\text{B1}} - 0.5| > 0.15$) collapses below that line under B2; across the 45 random-forest contrasts the median unpadded deviation of 0.137 drops to 0.038, and the per-family median Δ_{leak} is +0.173 (SPP), +0.058 (PCH), and −0.011 (PI, whose gating pair types are already at chance under B1). The clustering attacker shows the same pattern on a second, label-free attack family: the debiased NMI of the balanced PI pairs is 0.67 higher under B1 than under B2, and the DUO-term and logic-structure clusterings lose 0.35 and

0.10 of NMI, each with a policy-group bootstrap interval excluding zero. The effect is conditional, and we report it as such: the 12 random-forest contrasts with no detectable effect sit in the lower-left of the figure, where B1 is already near chance, and the 6 contrasts on which padding *raises* the deviation ($\Delta_{\text{leak}} < 0$, red) are cases where B1 is at chance and the padded transcript's remaining deviation is the per-policy pad-target draw examined in Sections 6.4 and 6.6 (ICD ch5 and `nestingDepth medium/deep` among them). These per-contrast counts are descriptive, not multiplicity-corrected; the causal claim rests on the direction and magnitude of the shift where a signal is present, corroborated by the independent clustering attacker.

What the un-padded attacker exploits. The B1 result deserves a precise reading, because the fixed circuit already makes the policy logic invisible to gas: the Groth16 verifier performs the same work for every proof, and the access-control path is the same for every policy. The un-padded access gas of the 1119 executions spans only 723,288–723,564 gas (one cold-start first call at 741,276 aside), in steps of 12 gas, and the ten ICD-10 chapter tasks do not separate on it. Twelve gas is the EIP-2028 price difference between a zero and a non-zero calldata byte, and decoding the calldata of the transactions shows where the variation comes from: the requester pseudonym id_{req}. In the evaluation harness, a requester who presents a profile credential is identified by a Poseidon-derived pseudonym that fills the 256-bit word, whereas a requester of an ontology-only policy carries a 128-bit random pseudonym whose upper 16 bytes are zero; the 192-gas difference between the two encodings is exactly the gap between the two clusters of B1 gas values, and whether a policy activates the profile predicate is correlated with its DUO terms, its structure type, and its parameter bucket. The remaining ± 84 gas come from the zero-byte counts of the proof elements, the policy and request identifiers, and the epoch, none of which depends on the policy. The un-padded leak is therefore not the policy logic but a sub-kilogas, deterministic calldata-pricing artifact of how one public transaction argument is encoded; it is a real channel for the observer nonetheless, and it is the class of channel that padding is designed to absorb, because randomizing the target by $\pm 10\%$ ($\pm 105{,}000$ gas) submerges any deterministic difference of a few hundred gas regardless of its origin. Encoding every pseudonym at full width would close this particular channel without padding, but not the class; the July 2026 predecessor of this ablation, run with an earlier circuit, separated structural buckets on a 140-gas spread that traced to identifier magnitudes and execution order in the same way.

Cost. Padding is not free: over the subset it raises mean commit gas from 219,737 (B1) to 304,098 (B2), a 38.4% increase, and mean access gas from 723,462 to 1,056,674, a 46.1% increase. The padded access cost is set by the padding band, not by the policy: the TPS draws the target uniformly from 945,000–1,155,000 gas, and the base of that band is chosen above the bare access cost measured on the target network (748,000 gas on Amoy, whose pricing of the pairing precompiles is higher than the local EVM's) so that the whole band stays above real usage. At the 30 gwei tip used throughout, one padded access transaction costs about 0.032 POL and one commit about 0.009 POL; the overhead is therefore a fixed per-transaction premium of roughly 330,000 gas on access and 85,000 on commit, independent of transaction volume, and it does not grow with policy complexity, because neither the bare cost nor the band depends on the policy. Table 15 gives the single-point cost characterization of the final RC1 configuration at $\mathsf{DEPTH} = 64$. The circuit compiles to 81,740 R1CS constraints; across the 1,773 Polygon Amoy executions, mean witness and proving times on the two-core capture host are 763.6 ms and 2,884.2 ms, respectively. Gas, logs, calldata lengths, and transaction objects are taken from the same executed dataset, whose transactions use the context-bound five-signal statement. Table 16 shows how the prover-side cost scales when the program length grows from $\mathsf{DEPTH} = 32$ to 128 with N

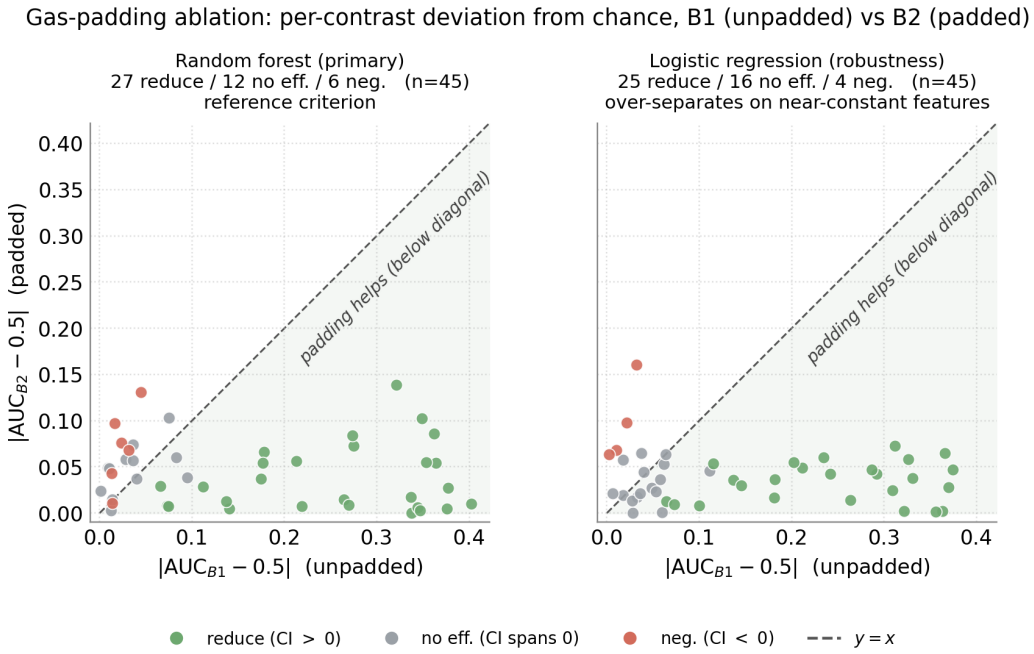


Figure 3. Gas-padding ablation over all 45 supervised contrasts: unpadded versus padded deviation from chance, colored by the sign of the 95% interval of Δ_{leak}.

and M fixed: constraints, proving-key size, and proving time grow roughly linearly with DEPTH (from 65,000 to 134,000 constraints and from 0.38 to 0.73 s per proof on a laptop-class prover), while the statement keeps its five public signals, so the verifier contract, its gas, and the padding band are unchanged. Richer predicates would enter through M and the per-slot gadgets rather than through DEPTH; each additional slot adds its gadget's constraints to the prover and nothing to the on-chain cost, since predicate parameters are bound through the fixed-format parameter commitment rather than exposed as public inputs.

7. Discussion

The evaluation supports a scoped claim: against the modeled public-chain metadata adversary, TPS-ZKPI provides empirical resistance rather than proven policy confidentiality. Of the 45 supervised tasks, 43 stay below the multi-seed leakage criterion, and unsupervised clustering shows no recoverable structure beyond one small, permutation-significant nesting-depth ARI. The two tasks that meet the criterion—one ICD-10 chapter and the `nestingDepth medium/deep` contrast—are reported as failures, and both are carried by the random pad target rather than by the policy: subtracting the logged target from the access gas returns each to PASS, and replacing the gas features by policy-independent draws reproduces such failures in four of ten replicates (Sections 6.2–6.6). This is a property of the criterion on a corpus in which coverage-layer policies are executed once, so the thirty seeds resample the same random draws; we keep the criterion as stated and document the diagnostic, since lowering the significance threshold or raising the AUC threshold to absorb these cases would weaken it for genuine leaks. The gas-padding ablation (Section 6.9) attributes the gas-channel protection to randomized gas padding: without it the same attacks recover the policy family with AUCs up to 0.90. Content hiding itself rests on the commitment-only interface and the content-independent transcript normalization; padding is the defense-in-depth layer that absorbs whatever deterministic gas differences remain, of which the ablation exhibits one that does not come from the policy logic at all but from the byte encoding of a public transaction argument. To-

Table 14. Gas-padding ablation on Polygon Amoy: B1 (no padding) versus B2 (deployed randomized padding) on the same 1119-execution subset; supervised rows are the largest-signal contrasts per experiment (random-forest AUC).

Attack	Target	B1 (unpad.)	B2 (pad.)
Supervised (RF AUC; Δ_{leak} with 95% interval)			
SPP	`structureType trivial/and_or_mixed`	0.902	0.490
	$\Delta_{\text{leak}} = 0.347$ CI$[0.334, 0.360]$		
SPP	`predicateCombo single/few`	0.844	0.494
	$\Delta_{\text{leak}} = 0.301$ CI$[0.287, 0.314]$		
PCH	DUO HMB	0.876	0.495
	$\Delta_{\text{leak}} = 0.337$ CI$[0.319, 0.352]$		
PI	`controlled_difference` †	0.676	0.446
	$\Delta_{\text{leak}} = 0.088$ CI$[0.062, 0.111]$		
Unsupervised (NMI, B1−B2 with policy-group bootstrap 95% interval)			
PCH	DUO terms	0.349 $[0.316, 0.407]$	
SPP	logic structure	0.102 $[0.018, 0.138]$	
PI	balanced pairs (debiased NMI)	0.670 $[0.591, 0.743]$	
Gas cost (mean over the subset, B1 → B2)			
Commit gas		219,737	304,098 (+38.4%)
Access gas		723,462	1,056,674 (+46.1%)

† Positive control: the least similar pairs by construction, excluded from the PI verdict.

Table 15. Single-point cost profile of the final RC1 configuration at DEPTH = 64. Gas, logs, calldata, and local prover timings are summarized over the 1,773 revised Polygon Amoy executions.

Metric	Value / source
Circuit size	81,740 R1CS constraints; 80,884 wires; DEPTH = 64
Public interface	4 public inputs plus 1 output
Padded on-chain gas	Commit 304,167; access 1,057,283 mean gas
Event logs	Commit: 1 log / 180 bytes; access: 2 logs / 296 bytes
Calldata length	Commit 196 bytes; access 612 bytes (constant across policies)
Local witness time	763.6 ms mean
Local prove time	2,884.2 ms mean

Table 16. Prover-side cost as the policy program length DEPTH grows, with $N = 13$, $M = 11$ and the public interface fixed. On-chain verification cost does not change: the statement keeps five public signals at every depth.

DEPTH	R1CS constraints	Proving key (MB)	Witness (ms)	Prove (ms)	Verify (ms)
32	64,945	38.7	282	382	11
64	81,740	52.2	343	494	10
128	134,009	87.5	542	730	11

gether with fixed-length policy representation, reusable interpretation, and commitment binding, these mechanisms limit the extent to which public-chain observers infer hidden policy content or structural properties from authorization transactions.

The scope of this claim is the adversary of Section 6.1: a passive observer of commitments, proofs, public parameters, calldata, events, and gas, on a corpus of generated policies. Timing and ordering of transactions, mempool and broadcast behaviour, off-chain communication with the TPS, adaptive requester probing, repeated access attempts, correlations between authorization outcomes and policy classes (every request in the corpus is granted), and compromised TPS behaviour are not modeled, and the results should not be read as establishing general policy confidentiality against them.

The cost of padding. Within this scope, the residual cost is the randomized padding's gas overhead: on the ablation subset, about 46% on access gas and 38% on commit gas (Section 6.9). Three observations bound its practical weight. First, the overhead is a fixed premium per transaction—roughly 330,000 gas on access and 85,000 on commit at the band used here—so it scales linearly with transaction volume and does not compound: at the 30 gwei tip of the evaluation, a padded access transaction costs about 0.032 POL against 0.022 POL unpadded. Second, neither the bare cost nor the band depends on the policy: the fixed circuit gives every proof the same verification work and the same five public signals, and the prover-side cost of longer or richer policies (Table 16) is paid off-chain. Third, the band must be positioned above the bare cost on the target network, which differs between networks (the same verifier costs 243,000 gas on a local EVM and 372,000 on Amoy, whose pairing precompiles are priced higher); a deployment therefore calibrates the base target once per network, and the width of the band, $\pm 10\%$ here, is what determines how large a deterministic difference it can submerge. Padding cannot, of course, hide a difference larger than its band, and it does not address channels that are not gas. The TPS trust assumption is the main architectural tradeoff. Moving confidentiality-sensitive proof generation to the TPS keeps policy-bound proving information away from requesters and public smart contracts while preserving on-chain verifiability, but it shifts part of the confidentiality burden to the service layer; the provider-operated TPS used here concentrates that burden at the provider rather than at an independent service (Section 4). This tradeoff leaves on-chain verifiability intact but does affect liveness and accountability: the TPS or provider can refuse to produce a proof, and such a refusal may be indistinguishable from a legitimate denial at the on-chain interface.

Attribute origin is only partially delegated to the TPS. For the profile-backed DUO terms, the circuit verifies the EdDSA-Poseidon signature described in Section 5 and derives eight policy attributes from the seven signed claims. A compromised TPS cannot modify the signed claims, policy commitment, requester pseudonym, resource, validity interval, or credential identifier without invalidating the signature. The implemented relation rejects a non-committed issuer, an invalid signature under the committed key, an expired or premature credential, and substitutions of any signed field. Other evidence has a narrower meaning: ontology proofs establish that a supplied code belongs to a provider-committed set, and requester allowlists establish that a holder pseudonym belongs to a provider-committed list; they do not authenticate every underlying real-world fact. Moreover, because the TPS receives the holder secret and credential witness in the current workflow, holder binding does not protect against a malicious TPS reusing otherwise valid evidence within its signed scope.

The remaining trust anchor is therefore issuer governance. The prototype binds the proof to a policy-selected issuer-key commitment, but it has no independent registry establishing that this key belongs to an accredited healthcare authority, and it does not check credential revocation. The evaluation uses a deterministic test-only issuer key to exercise

the complete signature path; it is not evidence of institutional issuance or production key management.

A concrete deployment can close this gap without changing the Boolean interpreter. First, a governed issuer registry would publish schema-qualified issuer keys and their validity epochs; registration would validate a canonical, subgroup-safe public key and require proof of possession. Second, an issuer would sign a domain-separated credential containing a holder-key commitment, claim vector, authorization scope, validity interval, and unique credential identifier. Third, the circuit would verify both the credential signature and a Merkle proof that the issuer key is active in the registry, while continuing to derive policy attributes from the signed claims. The holder would retain its secret key and authorize each request by signing the context digest and request epoch; verifying this second signature inside the circuit would prevent a TPS that observes one credential presentation from generating proofs for fresh sessions. Finally, an epoch-specific sparse-Merkle or accumulator root would support a private non-revocation witness for the credential identifier. The registry and revocation roots can occupy fixed-format public or committed parameter slots, preserving the policy-independent verification interface. If an existing healthcare credential uses a non-circuit-friendly signature such as Ed25519 or ECDSA, integration requires either verifying that signature in-circuit at higher constraint cost or having the issuer additionally issue a SNARK-friendly credential; re-signing by the TPS would not preserve issuer-origin assurance.

As described in Section 5, the policy commitment covers both the padded encoding P_{enc} and its active length ℓ. Because ℓ also determines how many tokens the interpreter evaluates, an authorizing proof for h cannot be based on a shorter valid prefix. This integrity guarantee is separate from our privacy evaluation, which measures whether public-chain artifacts reveal information about the hidden policy.

The authorization context is likewise part of the proved relation. The circuit constrains the public digest d_χ to the requester identifier, resource, and nonce used in the witness, and the ACM recomputes the same digest from the transaction parameters. Consequently, changing any of these values invalidates the request.

The guarantee is scoped to the bound tuple within one ACM deployment. The digest does not include the transaction submitter or a chain/contract domain, so exact-context front-running and cross-deployment reuse require separate deployment-level controls. Moreover, id_{req} is a pseudonymous authorization identifier and is not, by itself, proof of a real-world identity or of the origin of every private attribute.

Three directions follow. First, adding an explicit chain/contract domain and an authenticated submission channel would extend context binding beyond a single deployment and reduce front-running risk. Second, accountable proof obligations—for example, on-chain commitments to serve requests together with timeout-with-evidence mechanisms—would make proof refusals attributable. Third, the reduced-trust TPS instantiations of Section 4 (collaborative-SNARK or TEE-backed proving) would relax the trust concentration itself, including the attribute-origin limitation; realizing and evaluating them is left to future work.

8. Conclusion

This paper studied policy-private consent enforcement in blockchain-based healthcare data sharing under public-chain observability and an explicit TPS trust assumption. We proposed TPS-ZKPI, a Trusted Proof Service-assisted zero-knowledge approach that binds hidden provider-defined policies to on-chain commitments, evaluates Boolean-based consent policies through a reusable policy interpreter, and normalizes public verification artifacts to reduce metadata-level leakage. The revised construction closes two

soundness gaps of the original prototype: the program length is part of the committed and proved relation, so an authorizing proof cannot rest on a shorter valid prefix, and the proof is bound to the requester pseudonym, resource, and session nonce, so a captured proof cannot be replayed under fresh parameters within a deployment. Profile-backed data-use terms are derived in-circuit from an issuer-signed credential, which separates what the proof establishes cryptographically from what remains a matter of issuer governance.

The evaluation shows what such a design can and cannot promise. On a live testnet corpus, a supervised attacker with policy-disjoint training data and an unsupervised clustering attacker recover neither policy content, nor policy identity, nor structure and parameters from the normalized transcript beyond what the padding's own random draws produce, and the paired ablation shows that the padding is what removes the last policy-correlated gas signal. The ablation also delivers a cautionary result: the un-padded channel it exposes is not the policy logic, which the fixed circuit already hides, but the byte-level pricing of one public transaction argument—a reminder that on a public chain any deterministic encoding difference that correlates with the policy is a channel, and that the defense must cover the class rather than the instances one anticipated.

The results hold for a metadata-only public-chain adversary and are empirical. Policy confidentiality against the TPS itself is not cryptographic but rests on the service following its workflow, and the evaluation's provider-operated TPS concentrates that trust at the provider. The supported policy class is an access-time checker for bounded Boolean policies over standardized terms: purpose limitation, post-access duties, delegation, and credential revocation stay outside the proved statement. Timing, ordering, mempool, and off-chain channels, adaptive probing, and correlations between outcomes and policy classes are not modeled, and the leakage criterion has a measurable false-positive rate on corpora that execute each policy once.

Three directions follow. Reduced-trust proof generation—collaborative SNARKs or attested execution—would relax the trust concentration that the TPS introduces and, with a holder-signed authorization context, remove the service's ability to reuse valid evidence. A governed issuer registry with in-circuit membership and revocation witnesses would turn the credential path into an institutional guarantee. And extending the evaluation to active adversaries and to the timing and ordering channels excluded here would test whether transcript normalization remains sufficient once the observer is no longer passive.

Author Contributions: TODO.

Funding: TODO.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: TODO.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

TPS-ZKPI	Trusted Proof Service-assisted Zero-Knowledge Policy Interpreter
TPS	Trusted Proof Service
ACM	Access-Control Manager
ZKP	Zero-Knowledge Proof
SNARK	Succinct Non-interactive Argument of Knowledge
TEE	Trusted Execution Environment
RPN	Reverse Polish Notation
PCH	Policy Content Hiding
PI	Policy Indistinguishability
SPP	Structure and Parameter Privacy
DUO	Data Use Ontology
ICD-10	International Classification of Diseases, 10th Revision
OBI	Ontology for Biomedical Investigations
GA4GH	Global Alliance for Genomics and Health
CP-ABE	Ciphertext-Policy Attribute-Based Encryption
PRE	Proxy Re-Encryption
AUC	Area Under the (ROC) Curve
ARI	Adjusted Rand Index
NMI	Normalized Mutual Information
LR	Logistic Regression
RF	Random Forest

References

1. Kakarlapudi, P.V.; Mahmoud, Q.H. A systematic review of blockchain for consent management. In Proceedings of the Healthcare. MDPI, 2021, Vol. 9, p. 137.
2. Welzel, C.; Ostermann, M.; Smith, H.L.; Minssen, T.; Kirsten, T.; Gilbert, S. Enabling secure and self determined health data sharing and consent management. *npj Digital Medicine* **2025**, *8*, 560.
3. Li, K.; Sai, A.R.; Urovi, V. Do you need a blockchain in healthcare data sharing? A tertiary review. *Exploration of Digital Health Technologies* **2024**, *2*, 101–123.
4. Béres, F.; Seres, I.A.; Benczúr, A.A.; Quintyne-Collins, M. Blockchain is watching you: Profiling and deanonymizing ethereum users. In Proceedings of the 2021 IEEE international conference on decentralized applications and infrastructures (DAPPS). IEEE, 2021, pp. 69–78.
5. Li, K.; Lohachab, A.; Dumontier, M.; Urovi, V. Privacy preservation in blockchain-based healthcare data sharing: A systematic review. *Peer-to-Peer Networking and Applications* **2025**, *18*, 1–53.
6. Lawson, J.; Cabili, M.N.; Kerry, G.; Boughtwood, T.; Thorogood, A.; Alper, P.; Bowers, S.R.; Boyles, R.R.; Brookes, A.J.; Brush, M.; et al. The Data Use Ontology to streamline responsible access to human biomedical datasets. *Cell Genomics* **2021**, *1*.
7. Rondelet, A.; Zajac, M. Zeth: On Integrating Zerocash on Ethereum. arXiv:1904.00905, 2019, [arXiv:cs.CR/1904.00905].
8. Williamson, Z.J. The AZTEC Protocol. https://github.com/AztecProtocol/AZTEC, 2018. Technical whitepaper. Accessed: 2026-09-20.
9. Nadler, M.; Schär, F. Tornado cash and blockchain privacy: a primer for economists and policymakers. *Federal Reserve Bank of St. Louis Review* **2023**.
10. Ibraimi, L.; Petkovic, M.; Nikova, S.; Hartel, P.; Jonker, W. Ciphertext-policy attribute-based threshold decryption with flexible delegation and revocation of user attributes. Technical Report TR-CTIT-09-12, Centre for Telematics and Information Technology, University of Twente, 2009.
11. Ullah, S.S.; Oleshchuk, V.; Pussewalage, H.S.G. A survey on blockchain envisioned attribute based access control for internet of things: Overview, comparative analysis, and open research challenges. *Computer Networks* **2023**, *235*, 109994.
12. Zhou, L.; Diro, A.; Saini, A.; Kaisar, S.; Hiep, P.C. Leveraging zero knowledge proofs for blockchain-based identity sharing: A survey of advancements, challenges and opportunities. *Journal of Information Security and Applications* **2024**, *80*, 103678.
13. Hu, R.; Ma, Z.; Li, L.; Zuo, P.; Li, X.; Wei, J.; Liu, S. An access control scheme based on blockchain and ciphertext policy-attribute based encryption. *Sensors* **2023**, *23*, 8038.
14. World Health Organization. International Statistical Classification of Diseases and Related Health Problems (10th Revision), 2019. Accessed: 2026-07-08.
15. Dyke, S.O.; Philippakis, A.A.; Rambla De Argila, J.; Paltoo, D.N.; Luetkemeier, E.S.; Knoppers, B.M.; Brookes, A.J.; Spalding, J.D.; Thompson, M.; Roos, M.; et al. Consent codes: upholding standard data use conditions. *PLoS genetics* **2016**, *12*, e1005772.

16. Bethencourt, J.; Sahai, A.; Waters, B. Ciphertext-policy attribute-based encryption. In Proceedings of the 2007 IEEE symposium on security and privacy (SP'07). IEEE, 2007, pp. 321–334.
17. Ateniese, G.; Fu, K.; Green, M.; Hohenberger, S. Improved proxy re-encryption schemes with applications to secure distributed storage. *ACM Transactions on Information and System Security (TISSEC)* **2006**, *9*, 1–30.
18. Pei, H.; Yang, P.; Li, W.; Du, M.; Hu, Z. Proxy Re-Encryption for Secure Data Sharing with Blockchain in Internet of Medical Things. *Computer Networks* **2024**, *245*, 110373. https://doi.org/10.1016/j.comnet.2024.110373.
19. Zhou, H.; Si, X.M.; Zhao, B.; Zhang, Y.; Qu, Y. Secure electronic medical records sharing scheme based on blockchain by using proxy re-encryption. *Peer-to-Peer Networking and Applications* **2024**, *17*, 4197–4209. https://doi.org/10.1007/s12083-024-01775-y.
20. Yang, X.; Liu, J. HA-Med: A Blockchain-Based Solution for Sharing Medical Data with Hidden Policies and Attributes. *IET Information Security* **2024**, *2024*, 2498245. https://doi.org/10.1049/2024/2498245.
21. Ding, X.; Liu, Y.; Ning, J.; Chen, D. Blockchain-Enhanced Anonymous Data Sharing Scheme for 6G-Enabled Smart Healthcare With Distributed Key Generation and Policy Hiding. *IEEE Journal of Biomedical and Health Informatics* **2026**, *30*, 666–675. https://doi.org/10.1109/JBHI.2025.3550261.
22. Wu, N.; Xu, L.; Zhu, L. A blockchain based access control scheme with hidden policy and attribute. *Future Generation Computer Systems* **2023**, *141*, 186–196.
23. Groth, J. On the Size of Pairing-Based Non-interactive Arguments. In Proceedings of the Advances in Cryptology – EUROCRYPT 2016. Springer, 2016, pp. 305–326.
24. Sasson, E.B.; Chiesa, A.; Garman, C.; Green, M.; Miers, I.; Tromer, E.; Virza, M. Zerocash: Decentralized anonymous payments from bitcoin. In Proceedings of the 2014 IEEE symposium on security and privacy. IEEE, 2014, pp. 459–474.
25. Boneh, D.; Sahai, A.; Waters, B. Functional encryption: Definitions and challenges. In Proceedings of the Theory of Cryptography Conference. Springer, 2011, pp. 253–273.
26. Secret Network. Secret Network Overview. https://docs.scrt.network/secret-network-documentation/overview-ecosystem-and-technology/secret-network-overview. Accessed: 2026-06-29.
27. Cheng, R.; Zhang, F.; Kos, J.; He, W.; Hynes, N.; Johnson, N.; Juels, A.; Miller, A.; Song, D. Ekiden: A platform for confidentiality-preserving, trustworthy, and performant smart contracts. In Proceedings of the 2019 IEEE European Symposium on Security and Privacy (EuroS&P). IEEE, 2019, pp. 185–200.
28. Buccafurri, F.; De Angelis, V.; Nardone, R. How can the holder trust the verifier? A CP-ABPRE-based solution to control the access to claims in a Self-Sovereign-Identity scenario. *Blockchain: Research and Applications* **2024**, *5*, 100196. https://doi.org/10.1016/j.bcra.2024.100196.
29. Madine, M.; Salah, K.; Jayaraman, R.; Yaqoob, I. Zero-knowledge proofs for anonymous authentication of patients on public and private blockchains. *Array* **2025**, *28*, 100590. https://doi.org/10.1016/j.array.2025.100590.
30. Shashidhara, R.; Chirakarotu Nair, R.; Kumar Panakalapati, P. Promise of Zero-Knowledge Proofs (ZKPs) for Blockchain Privacy and Security: Opportunities, Challenges, and Future Directions. *Security and Privacy* **2024**, *8*, e461. https://doi.org/10.1002/spy2.461.
31. Ardagna, C.A.; De Capitani di Vimercati, S.; Paraboschi, S.; Pedrini, E.; Samarati, P. An XACML-based privacy-centered access control system. In Proceedings of the Proceedings of the first ACM workshop on Information security governance, 2009, pp. 49–58.
32. Sheikhalishahi, M.; Tillem, G.; Erkin, Z.; Zannone, N. Privacy-preserving multi-party access control. In Proceedings of the Proceedings of the 18th ACM Workshop on Privacy in the Electronic Society, 2019, pp. 1–13.
33. Gabizon, A.; Williamson, Z.J.; Ciobotaru, O. Plonk: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. *Cryptology ePrint Archive* **2019**.
34. Campanelli, M.; Fiore, D.; Querol, A. LegoSNARK: Modular design and composition of succinct zero-knowledge proofs. In Proceedings of the Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, 2019, pp. 2075–2092.
35. Liu, T.; Zhang, Z.; Zhang, Y.; Hu, W.; Zhang, Y. Ceno: Non-uniform, segment and parallel zero-knowledge virtual machine. *Journal of Cryptology* **2025**, *38*, 17.
36. Miller, T.; Alvarez, B.; Hoang, T. VTBC: Privatizing the Volume and Timing of Transactions for Blockchain Applications. In Proceedings of the 2023 32nd International Conference on Computer Communications and Networks (ICCCN). IEEE, 2023, pp. 1–10.
37. Bandrowski, A.; Brinkman, R.; Brochhausen, M.; Brush, M.H.; Bug, B.; Chibucos, M.C.; Clancy, K.; Courtot, M.; Derom, D.; Dumontier, M.; et al. The Ontology for Biomedical Investigations. *PLoS ONE* **2016**, *11*, e0154556.
38. Jaccard, P. The Distribution of the Flora in the Alpine Zone. *New Phytologist* **1912**, *11*, 37–50.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.

https://doi.org/10.3390/blockchains1010000